

Chaos labs

Theory, evidence and controlled experiments

Read the relevant theory, predict, measure, explain, and check recovery. Each lab tests one question. The solutions are collected at the end so you can study and run the question without seeing its expected observations.

Run `./lab.sh` provision once. Choose any lab: `./lab.sh NN` setup prepares and verifies its fixture in the shared VM. Open `./lab.sh ssh`, run the procedure and record your evidence. Compare NN solution. Save results before NN reset removes that lab's resources and files.

Stop if the baseline fails. Confirm the fault actually reached its target. Expected results are not your observations. Use Bash without `set -e` and preserve the same shell when blocks share variables. Keep a second VM terminal available for recovery.

All labs use one VM. Each Kubernetes lab owns a separate named kind cluster inside it. No lab sequence is required. Reset finished labs to free resources. `./lab.sh stop` halts the whole VM while preserving files. See README.md for capacity.

Long command lines wrap visually in this print edition. Copy exact commands from docs/chaos-labs.md or the terminal question card. Theory citations use original section numbers in docs/chaos-theory.md.

Lab	Topic	Lab	Topic
00	Prepare the learning environment	11	Health depends on what you test
01	A timeout makes a silent failure manageable	12	Locate startup failures and measure recovery
02	Prove why a process was killed	13	Placement, supervision and replacement
03	Restart policies have limits	14	Quorum, convergence and application availability
04	CPU contention and a quota	15	Count how layered retries multiply work
05	A container is several Linux mechanisms	16	Design one clear experiment
06	Separate containers can share a full filesystem	17	Signal delivery and the shutdown budget
07	Network delay accumulates across sequential calls	18	Persistence, ownership and read-only mounts
08	A syscall error tests application handling	19	Unscheduled Pods versus memory-killed containers
09	A syscall denial need not crash the process	20	Trace DNS, Service ports and the application listener
10	Ownership, disruption budgets and Service routing	21	Configuration freshness and a stalled rollout

Lab 00 - Prepare the learning environment

Which checks establish a usable baseline before you inject a fault?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 00 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A chaos experiment compares a measured outcome before, during and after one controlled fault. Observability means you can measure that outcome; a baseline is its normal value or range. If the no-fault check fails, stop: a later failure would not tell you whether the injection caused it.

The method is: choose a measurement, establish normal behaviour, predict a result under a specified fault, run the comparison, and explain the evidence. For example, “the client still responds within one second when its cache is unavailable” is testable; “the service is resilient” is not.

Blast radius is everything the experiment can affect. These labs target named processes, containers or objects inside a disposable Linux VM. Namespaces separate resource views; cgroup v2 accounts for and limits resource use. The VM still shares CPU, memory and a kernel across labs.

Record versions so another run can be compared with yours. Keep evidence in this lab's workspace. Reset deletes that workspace; shared tooling and other labs remain available. Each later experiment must still measure its own service baseline and recovery.

Source: docs/chaos-theory.md: §§1.3, 2.1, 2.5; Appendix A.

The experiment: Check the shared Ubuntu VM and save a baseline record inside Lab 00. No fault is injected.

How to read the evidence

Signal	Meaning
cgroup2fs / cgroups=2	The kernel and Docker use the cgroup v2 interface used by these labs.
hello-world	A successful run checks image access, container creation and process execution together.
df -h /	Size is total filesystem capacity; Avail is unused space. The shared provisioner requires at least 60 GiB total.
versions / baseline.txt	Record the actual versions and baseline checks, not simply “setup passed”.

Predict

If a prerequisite already fails, can a later failure tell you anything about the injected fault?

Record your results

Check	Observed value	Pass or action needed
cgroup / Docker		
Container execution		
Disk capacity		
Versions / baseline record		

Procedure

1. Check the kernel, cgroup, Docker and disk baseline

VM terminal 1

```
uname -m
stat -fc %T /sys/fs/cgroup
docker info --format 'driver={{.CgroupDriver}} cgroups={{.CgroupVersion}}'
docker run --rm --name ce-lab00-check hello-world
df -h /
```

2. Record the source commit, versions and image digests

VM terminal 1

```
cat ~/labs/book-commit.txt ~/labs/versions.txt
for image in python:3.12-slim ubuntu:24.04 busybox:1.36 nginx:1.27 ghcr.io/shopify/toxiproxy:2.12.0
  bloomberg/goldpinger:3.11.3; do
  printf '%s ' "$image"
  docker image inspect "$image" --format '{{json .RepoDigests}}'
done
```

3. Save the baseline record

VM terminal 1

```
{ cat ~/labs/book-commit.txt ~/labs/versions.txt; stat -fc %T /sys/fs/cgroup; docker info --format
'cgroups={{.CgroupVersion}}'; df -h /; } > ~/labs/lab00/baseline.txt
cat ~/labs/lab00/baseline.txt
```

Recovery check: Prerequisite checks pass and baseline.txt records them.

Answer

Name the checks that passed, the recorded environment version and the saved baseline record. Explain why a failed baseline blocks a meaningful comparison.

Check your understanding

Why must Lab 01 check Redis again even when hello-world succeeds?

Save results before removing this lab's resources and files: `./lab.sh 00 reset`

Lab 01 - A timeout makes a silent failure manageable

Why does the exception handler run after refusal but not while packets are silently dropped?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 01 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A dependency call must return or raise an error before execution can reach the next statement or an exception handler. `try/except` supplies a reaction to an error; it does not give the call a deadline.

REJECT with a TCP reset gives the client a prompt refusal. DROP silently discards matching packets. Without a reply the network stack retries and waits; its eventual failure can be much later than a useful application deadline. These are two different failure modes of the same dependency.

A connect timeout bounds connection establishment. A read timeout bounds waiting for data after connecting. The prepared client disables retries and sets both to `TIMEOUT` when supplied. A socket timeout is not a general end-to-end deadline across many operations or retries.

The five-second timeout command is an external watchdog: it terminates the test process. It does not execute the client's fallback. Compare that termination with the client catching its own timeout, then remove the firewall rule and check the same request again.

Source: docs/chaos-theory.md: §1.5 and Experiment Card 1.1.

The experiment: One ce-lab01 firewall rule targets TCP 127.0.0.1:6381. All fault runs use a five-second watchdog; TIMEOUT=0.5 sets the client timeouts. The comparison runs in ce-lab01-runner.service so reset can stop it before removing its firewall rules.

How to read the evidence

Signal	Meaning
OK / DEGRADED	OK means Redis answered. DEGRADED means the client caught a Redis error and executed its fallback.
elapsed	Seconds measured inside the client. A watchdog-killed, buffered process may print no elapsed line.
exit=124	The external timeout command reached its deadline. This is not the application's handled error.
iptables pkts	A nonzero counter confirms traffic matched the fault rule; no matches means you have not tested that fault.

Predict

Predict which of the three attempts reaches the handler: REJECT, DROP, and DROP with a 0.5-second timeout.

Record your results

Attempt	Handler ran?	Elapsed / exit	What ended the wait?
Baseline			
REJECT			
DROP			
DROP + 0.5 s timeout			
Recovered			

Procedure

1. Check the Redis baseline

VM terminal 1

```
cd ~/labs/lab01
redis-cli -p 6381 ping
python3 client.py
```

2. Predict, run, then prove recovery

VM terminal 1

```
cd ~/labs/lab01
sudo systemd-run --unit=ce-lab01-runner --collect --wait --pipe \
  --uid="$(id -u)" --property=RuntimeMaxSec=20s --working-directory="$PWD" \
  /bin/bash "$PWD/run.sh"
python3 client.py
sudo iptables -S OUTPUT | grep ce-lab01; echo "grep exit=$? (1 means no lab rule remains)"
```

Recovery check: The client succeeds again and no ce-lab01 rule remains.

Only if normal recovery fails

1. Manual rule removal, only if run.sh was killed

VM terminal 2

```
sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6381 -m comment --comment ce-lab01 -j DROP
sudo iptables -D OUTPUT -p tcp -d 127.0.0.1 --dport 6381 -m comment --comment ce-lab01 -j REJECT --reject-with
tcp-reset
python3 ~/labs/lab01/client.py
```

Answer

For each attempt, state whether the handler ran and what ended the wait. Use those observations to explain why catching errors alone does not bound waiting.

Check your understanding

If each attempt has a 0.5-second timeout and you allow three attempts, is the whole request bounded by 0.5 seconds?

Save results before removing this lab's resources and files: `./lab.sh 01 reset`

Lab 02 - Prove why a process was killed

Why is exit status 137 insufficient to diagnose an OOM kill?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 02 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A signal is a notification sent to a process. SIGTERM (15) can be handled for cleanup; SIGKILL (9) cannot be caught. Bash commonly reports signal termination as 128 + signal number, so a SIGKILL gives 137. A program may also explicitly exit with 137: status alone is not a diagnosis.

The kernel can invoke the out-of-memory (OOM) killer when it cannot satisfy memory demand. A cgroup has its own memory boundary: its processes can hit MemoryMax even while the rest of the VM has available memory. The allocator here repeatedly creates and touches bytes, so it demands real memory rather than merely reserving virtual addresses.

MemoryMax=128M and MemorySwapMax=0 constrain this experiment. RuntimeMaxSec=20s is a separate time bound. An OOM kill, that timer, and a deliberate SIGKILL can all stop the process; identify which happened by matching the unit, PID and time in the logs.

The `systemd-run` command reports the transient unit's outcome; its shell status need not equal the child's Bash status. Collect the unit result and kernel evidence. `systemd-oomd` is a userspace memory-pressure killer, so an `oomd` action is a different cause from a kernel OOM kill.

Source: docs/chaos-theory.md: §2.3 (exit codes, signals and OOM), §5.5.2 (memory limits).

The experiment: The allocator runs only in ce-lab02, with 128 MiB memory, no swap and a 20-second runtime bound.

How to read the evidence

Signal	Meaning
deliberate KILL exit	Read \$? immediately after the process. An intervening command would replace it.
journalctl -u ce-lab02	Look for the recorded termination signal and result: oom-kill and timeout describe different causes.
journalctl -k	Match a memory-cgroup OOM / Killed process line to this allocator and experiment time.
journalctl -u systemd-oomd	A matching userspace-kill entry changes the diagnosis; unrelated historical entries do not.

Predict

Will a deliberate SIGKILL and a memory-limit kill have different exit statuses? Which evidence can distinguish them?

Record your results

Case	Termination	Matching cause evidence
Known SIGKILL		
Memory-limited allocator		

Procedure

1. Record a deliberate SIGKILL

VM terminal 1

```
python3 -c 'import os,signal; os.kill(os.getpid(),signal.SIGKILL)'  
echo "deliberate KILL exit=$?"
```

2. Cause a cgroup-bounded OOM and collect unit plus kernel evidence

VM terminal 1

```
since=$(date --iso-8601=seconds)  
sudo systemd-run --unit=ce-lab02 --wait --collect \  
-p MemoryMax=128M -p MemorySwapMax=0 -p RuntimeMaxSec=20s \  
/usr/bin/python3 -c 'a=[]  
while True: a.append(b"x"*1000000)'  
sudo journalctl -u ce-lab02 --since "$since" --no-pager  
sudo journalctl -k --since "$since" --no-pager | grep -iE 'oom|killed process'  
sudo journalctl -u systemd-oomd --since "$since" --no-pager
```

3. Stop the allocator if it remains active

VM terminal 1

```
sudo systemctl stop ce-lab02.service 2>/dev/null || true
```

Recovery check: The allocator is stopped and its termination evidence has been recorded.

Answer

Compare the known SIGKILL with the allocator result. Cite the matching log that identifies the allocator's cause; status 137 alone is not enough.

Check your understanding

Does free memory elsewhere in the VM rule out a cgroup OOM kill?

Save results before removing this lab's resources and files: ./lab.sh 02 reset

Lab 03 - Restart policies have limits

Why does one crash recover while a burst of crashes can leave the same service failed?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 03 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A supervisor separates a service's desired state from its current process. `Restart=always` asks `systemd` to start a replacement after a crash. It is still subject to start-rate limiting; the request for a restart and permission to start are separate decisions.

This fixture explicitly sets `StartLimitIntervalSec=20` and `StartLimitBurst=4`. Starts consume the allowance, including initial or manual starts; successful uptime does not instantly erase it. A single kill after reset-failed can recover, whereas enough closely spaced starts exhaust the allowance and stop automatic recovery.

NGINX sends requests to A or B and can try another usable backend on a connection failure. If A gives up restarting, B may still answer. Measure A's state and client HTTP results independently: successful HTTP through the proxy cannot identify which backend served it.

reset-failed clears the failed state and start counter; start requests recovery. Disabling the limit would permit more restart attempts, but would not fix the cause of the crashes. A passing single-crash experiment therefore says little about a repeated-crash condition.

Source: docs/chaos-theory.md: §§2.4-2.6 and Experiment Cards 2.1-2.2.

The experiment: A and B serve through NGINX. The core experiment targets only A: `Restart=always`, at most four starts per 20 seconds. B stays available during this comparison.

How to read the evidence

Signal	Meaning
ActiveState / Result	ActiveState shows whether A is running or failed. Result and the journal explain a refused restart.
NRestarts	Counts automatic restarts; it is not the complete count of all starts used by the rate limiter.
Start request repeated too quickly	Evidence that the start-rate limit was reached. A later kill saying the unit is inactive is a consequence, not another successful injection.
HTTP response	A successful response tests the proxy path, not the health of both backends.

Predict

Predict A's state after one kill and after six rapid kill attempts, with `Restart=always` unchanged.

Record your results

Phase	A state / result	HTTP result	Journal evidence
Baseline			
One crash			
Crash burst			
Recovered			

Procedure

1. Check baseline and clear the previous start counter

VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a
systemctl is-active ce-lab03-a ce-lab03-b
curl -fsS --max-time 3 http://127.0.0.1:8003/
```

2. Kill A once and inspect its recovery

VM terminal 1

```
sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a
sleep 1
curl -fsS --max-time 3 http://127.0.0.1:8003/
systemctl show ce-lab03-a -p ActiveState -p NRestarts
```

3. Repeat crashes rapidly and inspect the refused restart

VM terminal 1

```
for i in $(seq 1 6); do
  sudo systemctl kill --kill-whom=main --signal=KILL ce-lab03-a || true
  sleep 1
done
systemctl show ce-lab03-a -p ActiveState -p Result -p NRestarts
sudo journalctl -u ce-lab03-a -n 15 --no-pager
curl -sS --max-time 3 -o /dev/null -w "proxy HTTP=%{http_code}\n" http://127.0.0.1:8003/
```

4. Recover and prove the proxy works again

VM terminal 1

```
sudo systemctl reset-failed ce-lab03-a ce-lab03-b
sudo systemctl start ce-lab03-a ce-lab03-b
systemctl is-active ce-lab03-a ce-lab03-b
# The backends need a moment to bind, and NGINX retries a failed backend after fail_timeout=1s.
for i in $(seq 1 10); do curl -fsS --max-time 3 http://127.0.0.1:8003/ && break; sleep 0.5; done
```

Recovery check: Both backends are active and the proxy responds successfully.

Answer

Compare A's state after one crash and after the burst. Quote the start-limit evidence and explain why Restart=always did not guarantee another start.

Check your understanding

What would disabling the start limit prove, and what defect would remain?

Save results before removing this lab's resources and files: ./lab.sh 03 reset

Lab 04 - CPU contention and a quota

How does capping a CPU competitor change the time taken by the same job?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 04 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A CPU-bound job needs CPU execution time. Its wall time also includes waiting to be scheduled. Keeping the calculation fixed while adding another runnable job on the same CPU tests contention without changing the application's work.

USE means utilization, saturation and errors, checked per resource. CPU utilization says how busy the CPU is; saturation is runnable work waiting for CPU. High utilization alone does not prove a problem. Connect the resource measurements to a change in this job's elapsed time.

CPU placement and CPU bandwidth are different controls. `taskset` pins both jobs to the same allowed CPU. `CPUQuota=20%` caps the neighbour at one fifth of one CPU on average. It neither reserves a core nor promises the remaining time exclusively to the measured job.

In `cgroup v2`, `cpu.max` contains quota and period in microseconds. Divide quota by period to obtain the CPU budget, for example $20000 / 100000 = 0.2$. When the group uses that budget it waits until replenishment; rising throttling counters show enforcement. Compare repeated baseline timings, loaded timings, capped timings and recovery using the same calculation.

Source: docs/chaos-theory.md: §§3.2, 3.3.5 and the CPU-shares correction; §5.5.2.

The experiment: Select one allowed CPU automatically and pin both jobs to it. The neighbour runs in `ce-lab04` for at most 45 seconds. Each measurement uses the same six-iteration job.

How to read the evidence

Signal	Meaning
work.py output	Column 1 is iteration number; column 2 is elapsed seconds. Mean is the sum of column 2 divided by the number of rows; range is its minimum to maximum. Compare all samples in each phase, not a single fast sample.
cpu.max / cpu.stat	<code>cpu.max</code> is configuration. <code>nr_throttled</code> counts throttled periods; <code>throttled_usec</code> is accumulated throttling time. Configuration alone does not show that a limit was reached.
CPU PSI / vmstat r	PSI reports time with runnable tasks waiting; <code>r</code> counts runnable tasks. Both are VM-wide supporting signals, not attribution to this one neighbour.
Neighbour active	If it ended before the job finished, the sample mixes loaded and recovered states and must be repeated.

Predict

Rank the job time with no competitor, an unrestricted competitor and a competitor capped at 20%. Explain your prediction.

Record your results

Phase	Mean job time (s)	Neighbour active?	Throttle evidence
Baseline			
Unrestricted neighbour			
20% neighbour			
Recovered			

Procedure

1. Choose an allowed CPU

VM terminal 1

```
cd ~/labs/lab04
CPU=$(python3 -c 'import os; print(min(os.sched_getaffinity(0)))')
echo "Both workloads will use CPU $CPU"
```

2. Record the baseline three times

VM terminal 1

```
cd ~/labs/lab04
for run in 1 2 3; do taskset -c "$CPU" python3 work.py | tee "baseline-$run.txt"; done
sort -k2 -n baseline-*.txt | awk 'NR==1 {min=$2} {max=$2} END {print "iteration range:", min, "-", max, "s"}'
```

3. Add one bounded CPU neighbour on the same CPU

VM terminal 1

```
cd ~/labs/lab04
sudo systemd-run --unit=ce-lab04 --collect -p RuntimeMaxSec=45s \
  taskset -c "$CPU" stress-ng --cpu 1 --timeout 40s
taskset -c "$CPU" python3 work.py | tee loaded.txt
systemctl is-active --quiet ce-lab04 || echo 'Inconclusive: the neighbour ended before the measurement finished.'
cat /proc/pressure/cpu
vmstat 1 5
sudo systemctl stop ce-lab04
```

4. Cap the neighbour at 20 percent and read throttling counters

VM terminal 1

```
cd ~/labs/lab04
for i in $(seq 1 10); do
  systemctl list-units --all --plain --no-legend ce-lab04.service | grep -q . || break
  sleep 1
done
sudo systemd-run --unit=ce-lab04 --collect -p CPUQuota=20% -p RuntimeMaxSec=45s \
  taskset -c "$CPU" stress-ng --cpu 1 --timeout 40s
taskset -c "$CPU" python3 work.py | tee quota.txt
path=$(systemctl show ce-lab04 -p ControlGroup --value)
if systemctl is-active --quiet ce-lab04 && [ -n "$path" ]; then
  sudo cat "/sys/fs/cgroup$path/cpu.max" "/sys/fs/cgroup$path/cpu.stat"
else
  echo 'Inconclusive: the neighbour ended before the measurement finished.'
fi
sudo systemctl stop ce-lab04
```

5. Recover and repeat the baseline

VM terminal 1

```
cd ~/labs/lab04
sudo systemctl stop ce-lab04 2>/dev/null || true
for i in $(seq 1 10); do
    systemctl list-units --all --plain --no-legend ce-lab04.service | grep -q . || break
    sleep 1
done
taskset -c "$CPU" python3 work.py | tee recovered.txt
for f in baseline-1.txt baseline-2.txt baseline-3.txt loaded.txt quota.txt recovered.txt; do
    awk -v f="$f" '{s+=$2} END {printf "%-15s mean=%.4fs\n", f, s/NR}' "$f"
done
```

Recovery check: ce-lab04 is stopped and the final job timing is recorded.

Answer

Compare the measured job times and quota counters. Explain how throttling the competitor changes waiting for CPU; mark a run inconclusive if the competitor stopped too early.

Check your understanding

Would pinning the neighbour to a different otherwise idle CPU test the same contention?

Save results before removing this lab's resources and files: `./lab.sh 04 reset`

Lab 05 - A container is several Linux mechanisms

Why does an isolated process view not tell you how much CPU a sandbox can use?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 05 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A Linux container combines mechanisms; it is not a small machine with its own kernel. chroot changes where absolute filesystem paths begin. A mount namespace separates mount configuration. A PID namespace gives processes their own PID numbering and visibility.

`unshare --pid --fork` starts a child in a new PID namespace. Its first process is PID 1 there but has an ordinary PID visible from the VM. A fresh `proc` mount is needed for `ps` to display the new view; changing the filesystem root alone does not hide host processes.

Cgroups separately account for and constrain resource consumption. The `systemd` scope places the sandbox in a group with `CPUQuota=20%`, `MemoryMax=128M` and `TasksMax=50`. An isolated `ps` listing says nothing about those budgets.

A CPU quota is time per period: divide the two `cpu.max` numbers. Read `cpu.stat` before and during a busy loop to distinguish a configured cap from actual throttling. The sandbox still shares the VM kernel and, because no network namespace is requested, its network. Each isolation claim needs its own evidence.

Source: docs/chaos-theory.md: §§5.2-5.5.2 and §5.7.1.

The experiment: The prepared BusyBox sandbox uses `chroot`, PID and mount namespaces, and a `systemd` scope with CPU, memory and task limits.

How to read the evidence

Signal	Meaning
echo \$\$ / ps	PID 1 and the small process list establish the sandbox's PID view. The host can still see these processes.
ls /	Shows the exported BusyBox filesystem used as the sandbox's root.
cpu.max / memory.max / pids.max	cpu.max gives quota and period in microseconds; $20000 / 100000 = 0.2$ of one CPU. memory.max is bytes; 128 MiB = 134217728 bytes. pids.max counts tasks, including threads.
cpu.stat	nr_throttled counts periods with throttling; throttled_usec accumulates throttled microseconds. Compare the later value minus the earlier value. An increase during the loop shows enforcement; an old nonzero total alone does not.

Predict

Predict which observation shows a separate PID view and which shows enforced CPU limiting.

Record your results

Observation	Value / change	Responsible mechanism
Filesystem view		
PID view		
Configured limits		
Busy-loop throttling		

Procedure

1. Confirm the exported BusyBox root filesystem

VM terminal 1

```
ls ~/labs/lab05/rootfs
```

2. Start the sandbox under a resource-limited systemd scope (this shell becomes the sandbox)

VM terminal 1

```
sudo systemd-run --unit=ce-lab05 --scope --collect \
  -p MemoryMax=128M -p MemorySwapMax=0 -p TasksMax=50 -p CPUQuota=20% \
  unshare --fork --pid --mount --mount-proc="$HOME/labs/lab05/rootfs/proc" \
  chroot "$HOME/labs/lab05/rootfs" /bin/sh
```

3. Inspect the view inside the sandbox

Sandbox shell (terminal 1)

```
echo $$
ps
ls /
```

4. Read the limits and PID namespaces from the host

VM terminal 2

```
path=$(systemctl show ce-lab05.scope -p ControlGroup --value)
sudo cat "/sys/fs/cgroup$path/cpu.max" "/sys/fs/cgroup$path/memory.max" "/sys/fs/cgroup$path/pids.max"
sudo lsns -t pid
```

5. Run a finite busy loop inside the sandbox

Sandbox shell (terminal 1); observe from terminal 2 while it runs

```
timeout 30 sh -c "while ;; do ;; done"
```

6. Observe enforcement from the host while the loop runs

VM terminal 2

```
path=$(systemctl show ce-lab05.scope -p ControlGroup --value)
sudo cat "/sys/fs/cgroup$path/cpu.stat"
sleep 5
sudo cat "/sys/fs/cgroup$path/cpu.stat"
top -b -n 2 -d 2 | awk '/^top -/ {frame++} frame == 2' | head -15
```

7. After the loop finishes, exit the sandbox

Sandbox shell (terminal 1)

```
exit
```

8. Confirm the scope stopped

VM terminal 2

```
systemctl is-active ce-lab05.scope; echo "status=$? (nonzero means not active)"
```

Recovery check: The sandbox is exited and its scope is no longer active.

Only if normal recovery fails

1. Fallback if the sandbox shell is unresponsive

VM terminal 2

```
sudo systemctl stop ce-lab05.scope
sudo systemctl reset-failed ce-lab05.scope 2>/dev/null || true
```

Answer

Use the PID view and increasing throttling counters to explain the separate roles of namespaces and cgroups.

Check your understanding

Would removing CPUQuota make host processes appear in the sandbox's ps output?

Save results before removing this lab's resources and files: `./lab.sh 05 reset`

Lab 06 - Separate containers can share a full filesystem

Why can one container prevent another from writing to a shared filesystem?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 06 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

Containers can see different root filesystems yet share the same storage capacity. A bind mount exposes a host directory inside a container; mounting that same directory into two containers does not make two copies of its free space.

This experiment uses a 32 MiB tmpfs, a small memory-backed filesystem, as the shared capacity pool. It demonstrates the shared-capacity mechanism from the book without filling the VM's root disk. It does not measure disk throughput or reproduce Docker image-layer storage.

A write needs both permission and available capacity. `dd` writes a known number of bytes; when the filesystem cannot hold more, it can write only part of the data and then return `ENOSPC` (“No space left on device”). A fresh container still sees the same full mount.

Use the identical one-MiB probe before filling, while full, and after deleting the filling file. A failure only in the full phase, supported by `df`, connects the symptom to shared capacity. CPU and PID cgroup limits would not create more capacity on this mount.

Source: docs/chaos-theory.md: §5.4 and Experiment Card 5.1. The `tmpfs` fixture adapts the shared-storage example.

The experiment: Two disposable containers mount the same 32 MiB `tmpfs`. Only this small lab mount is filled.

How to read the evidence

Signal	Meaning
<code>dd bs=1M count=1</code>	Attempts to write one MiB. Record the write's error directly. In the filling command, <code>df</code> runs after <code>dd</code> , so the container can exit zero even though <code>dd</code> failed.
<code>df -h /data</code>	Shows capacity of the shared mount. Avail approaching zero supports the <code>ENOSPC</code> explanation.
<code>--rm</code>	Removes the container after exit. It does not delete data written into the host bind mount.
mountpoint	A false result after recovery confirms the temporary filesystem was unmounted.

Predict

Will a fresh container be able to write after another fills their shared mount?

Record your results

Phase	Probe result	Available space / error
Empty		
Full		
Space freed		

Procedure

1. Mount the small shared filesystem and prove a write works

VM terminal 1

```
mkdir -p ~/labs/lab06/shared
sudo mount -t tmpfs -o size=32m tmpfs "$HOME/labs/lab06/shared"
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero of=/data/probe bs=1M count=1'
sudo rm -f ~/labs/lab06/shared/probe
df -h ~/labs/lab06/shared
```

2. Fill it from one container and repeat the same write from another

VM terminal 1

```
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero of=/data/full bs=1M count=40; df -h /data'
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero of=/data/probe bs=1M count=1'
```

3. Free the space, repeat the write, and unmount

VM terminal 1

```
sudo rm -f ~/labs/lab06/shared/full ~/labs/lab06/shared/probe
docker run --rm --name ce-lab06-writer -v "$HOME/labs/lab06/shared:/data" ubuntu:24.04 sh -c 'dd if=/dev/zero
of=/data/probe bs=1M count=1'
sudo rm -f ~/labs/lab06/shared/probe
df -h ~/labs/lab06/shared
sudo umount ~/labs/lab06/shared
mountpoint ~/labs/lab06/shared; echo "status=$? (nonzero means unmounted)"
```

Recovery check: No ce-lab06 test container remains and the shared tmpfs is unmounted.

Answer

Compare the identical write before filling, while full and after freeing space. Explain which resource the two containers share.

Check your understanding

Why does deleting the container that filled the mount not recover the space?

Save results before removing this lab's resources and files: `./lab.sh 06 reset`

Lab 07 - Network delay accumulates across sequential calls

How does the same network delay affect one dependency exchange versus four sequential exchanges?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 07 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A network namespace holds interfaces, routes and packet queueing configuration. `nsenter --net` starts a host command in a target process's network namespace. The command uses the host's installed `tc` tool while changing the container's `eth0`. No extra tool needs to be installed in the target image.

`tc` configures a queueing discipline (`qdisc`), the rule for scheduling outgoing packets on an interface. `netem delay 25ms` delays each outgoing packet by approximately 25 ms. Here only the dependency's egress is changed; the VM interface and the opposite direction are not independently delayed.

The prepared client sends one byte, waits for its reply, and repeats on the same TCP connection. If a request contains N sequential exchanges, added delay is approximately $N \times d$, with packet and scheduling variation. Timing starts after connecting, so connection establishment is excluded. Compare $N=1$ and $N=4$ after subtracting each case's own baseline. Sequential waits add; independent parallel exchanges can overlap, so their completion time follows the slowest required exchange.

The `tc` command exits after installing the rule. The rule remains in the network namespace until removed or the interface is destroyed. Inspect `qdisc` state and counters, measure a near-zero control and the larger delay, then delete the rule and repeat the original measurement.

Source: docs/chaos-theory.md: §§5.8-5.8.1 and Experiment Card 5.5.

The experiment: A Python echo server runs in ce-lab07 on port 8070. The host client measures one or four exchanges per sample. Change only the container's `eth0` egress: no delay -> 1 ms control -> 25 ms -> no delay.

How to read the evidence

Signal	Meaning
exchanges / mean_ms	Each sample is the elapsed time for N sequential echo exchanges after connecting. Five samples give a mean and range.
tc -s qdisc show	netem with the requested delay confirms configuration; Sent counters increasing after probes confirm traffic traversed it.
1 ms control	Checks the measurement path with a small injected dose. It includes that dose and timing noise, so it is not a pure measurement of tool overhead.
noqueue after deletion	The fault has been removed. The repeated client measurement checks service recovery separately.

Predict

Estimate the added time for one and four sequential exchanges at 25 ms per outgoing reply. Will exiting tc remove the delay?

Record your results

Delay	1 exchange mean (ms)	4 exchanges mean (ms)	qdisc / traffic evidence
None			
1 ms control			
25 ms			
Removed			

Procedure

1. Measure the unchanged dependency

VM terminal 1

```
cd ~/labs/lab07
net() { sudo nsenter --target "$(docker inspect -f '{{.State.Pid}}' ce-lab07)" --net -- tc "$@"; }
net qdisc show dev eth0
python3 probe.py 1
python3 probe.py 4
```

2. Measure a 1 ms control

VM terminal 1

```
net qdisc add dev eth0 root netem delay 1ms
python3 probe.py 1
python3 probe.py 4
net -s qdisc show dev eth0
```

3. Change only the dose to 25 ms

VM terminal 1

```
net qdisc change dev eth0 root netem delay 25ms
python3 probe.py 1
python3 probe.py 4
net -s qdisc show dev eth0
```

4. Remove the rule and repeat both probes

VM terminal 1

```
net qdisc del dev eth0 root
net qdisc show dev eth0
python3 probe.py 1
python3 probe.py 4
```

Recovery check: The container has no netem qdisc and both client measurements return toward their own baseline.

Only if normal recovery fails

1. Remove only this container's injected queue

VM terminal 2

```
PID=$(docker inspect -f '{{.State.Pid}}' ce-lab07)
if [ "$PID" -gt 1 ]; then
  sudo nsenter --target "$PID" --net -- tc qdisc del dev eth0 root
fi
```

Answer

Subtract each exchange count's baseline from its delayed mean. Compare with $N \times 25$ ms, cite qdisc traffic evidence, and explain why explicit deletion is needed for recovery.

Check your understanding

If four exchanges could run independently in parallel, would their delays necessarily add to four times the dose?

Save results before removing this lab's resources and files: `./lab.sh 07 reset`

Lab 08 - A syscall error tests application handling

Why can one request succeed even though its close error stops the server?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 08 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A syscall crosses the application/kernel boundary. `write` sends bytes through a file descriptor; `close` releases a descriptor. The application receives a result or error and decides whether to retry, continue or exit. A syscall error and a process exit status are different observations.

`strace -p` attaches to one running process. `-e trace=close` selects the displayed calls; `-e inject=close:error=EIO` skips the selected real calls and makes them report an I/O error. The INJECTED marker distinguishes this artificial error from a naturally occurring one. Attaching after startup avoids injecting into unrelated initialization calls.

The server's request path writes the response before closing the connection. Failure on a later close can therefore occur after the client received bytes. Test the triggering request, the process state, and the next request separately; a successful first response cannot establish that the service continues.

Observe a normal request with tracing before injecting. Tracing adds overhead, so this lab tests error handling rather than throughput. The normal trace may include an ignored `fsync` error: seeing an error does not prove that it caused the exit. Follow the actual sequence from injected call to server log to later availability.

On Linux, a real close can release the descriptor even when it reports an error. Skipping close with this injection tests the application's error path, not every real close side effect. Do not infer a descriptor leak

from the error code alone or blindly retry close on a descriptor number that may have been reused.

Source: docs/chaos-theory.md: §§6.2-6.4 and Experiment Cards 6.1-6.2.

The experiment: Target only the PID saved in server.pid. Use two VM terminals; Ctrl-C in the tracing terminal detaches strace.

How to read the evidence

Signal	Meaning
strace return / INJECTED	The syscall returns -1 with EIO. The marker confirms the injected path was reached.
HTTP code / size_download	Code is the response status and size_download is body bytes received. 000 means no HTTP status was obtained, not a server status code.
wait "\$server" / log	In the shell that started the child, wait reports its exit status. Correlate that with the "error closing socket" message.
Next request	Tests continued service after the triggering request; this is independent of whether the earlier response completed.

Predict

Predict the triggering response, process state and next request after close returns EIO.

Record your results

Phase	HTTP code / bytes	Process state / exit	Trace or log evidence
Normal traced request			
Injected request			
Next request			
Restarted			

Procedure

1. Start the compiled book server and record its exact PID

VM terminal 1

```
cd ~/labs/lab08
./legacy_server > ~/labs/lab08/server.log 2>&1 & server=$!
echo "$server" > server.pid
sleep 1
curl -fsS --max-time 3 http://127.0.0.1:8080/ | head -3
echo "server PID=$server"
```

2. Observe writes, closes and fsyncs without changing behaviour (Ctrl-C to detach)

VM terminal 2

```
PID=$(cat ~/labs/lab08/server.pid)
sudo strace -p "$PID" -e trace=write,close,fsync -o ~/labs/lab08/trace.txt
```

3. Request one page while the tracer is attached

VM terminal 1

```
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/
```

4. After Ctrl-C in terminal 2, inspect the trace

VM terminal 2

```
awk -F '(' '{print $1}' ~/labs/lab08/trace.txt | sort | uniq -c  
grep -E '= -1' ~/labs/lab08/trace.txt | head
```

5. Inject EIO into close (leave attached)

VM terminal 2

```
PID=$(cat ~/labs/lab08/server.pid)  
sudo strace -p "$PID" -e trace=close -e inject=close:error=EIO
```

6. Request one page and read the application outcome

VM terminal 1

```
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/  
sleep 1  
if kill -0 "$server" 2>/dev/null; then echo 'server still running'; else wait "$server"; echo "server exit=$?";  
fi  
tail -3 ~/labs/lab08/server.log  
curl -sS --max-time 3 -o /dev/null -w 'next request: %{http_code}\n' http://127.0.0.1:8080/
```

7. Restore HTTP service, then stop the test server

VM terminal 1

```
cd ~/labs/lab08  
if kill -0 "$server" 2>/dev/null; then kill "$server"; wait "$server"; fi  
./legacy_server > ~/labs/lab08/server.log 2>&1 & server=$!  
echo "$server" > server.pid  
sleep 1  
curl -sS --max-time 3 -o /dev/null -w '%{http_code} %{size_download}\n' http://127.0.0.1:8080/  
kill "$server"; wait "$server"  
rm -f ~/labs/lab08/server.pid
```

Recovery check: The server responds after restart, then the test process is stopped.

Answer

Connect the injected close error to the server log and exit status. Explain the first and next HTTP results separately.

Check your understanding

Does this test show that every real close error leaves a file descriptor open?

Save results before removing this lab's resources and files: `./lab.sh 08 reset`

Lab 09 - A syscall denial need not crash the process

What changes when the same process installs a filter that denies getpid?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 09 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

`seccomp` is a kernel filter on a process's syscall attempts. A filter can allow a call, return a chosen error, or use other actions. This example allows calls by default and adds one rule: `getpid` returns `EACCES`. It installs the policy inside its own process using `libseccomp`.

The program calls `syscall(SYS_getpid)` directly before and after loading the filter. That makes the kernel-boundary test explicit rather than relying on a library's implementation of `getpid`. Before filtering a successful call returns a positive PID; an error through `syscall` returns `-1` and sets `errno`.

Returning an error is not the same as killing the process. write and process-exit operations remain allowed, so the program can print the denied result and finish. Its exit status reports whether the demonstration's check passed; it is not the getpid return value.

A loaded filter applies to the test process and is inherited by descendants. Freeing the userspace libseccomp context does not remove the kernel policy. The process cannot simply undo the filter; ending it ends this experiment. A fresh process launched by the original shell does not inherit a filter installed only in the child.

Source: docs/chaos-theory.md: §§6.2 and 6.5.2 (syscalls and libseccomp).

The experiment: Setup compiles filter.c. Running ./filter changes only that test process's syscall policy. No container is required.

How to read the evidence

Signal	Meaning
getpid result	A positive number is a successful PID result. -1 indicates a syscall error; read errno with it.
errno=13	EACCES: the rule's chosen permission-denied error. Only interpret errno after an error return.
exit	0 means the expected denial was observed; 1-3 mean filter setup failed; 4 means the observation did not match the rule.
Seccomp	In /proc/self/status, 0 means disabled and 2 means filter mode. This reports the reading process's state, not proof of which syscall was denied.

Predict

Predict getpid's result before and after the filter, and whether the program can still print the error.

Record your results

Phase	Syscall result	errno / process exit
Before filter		
After filter		
Fresh process		

Procedure

1. Run the prepared seccomp demonstration

VM terminal 1

```
cd ~/labs/lab09
./filter; echo "exit=$?"
```

2. Check an unrelated process after the test exits

VM terminal 1

```
grep Seccomp /proc/self/status
```

Recovery check: The demonstration has exited; the filter was scoped to that process. Reset removes the compiled fixture.

Answer

Explain the positive PID before filtering, the error afterward and why exit 0 means the demonstration detected the intended denial.

Check your understanding

Would `seccomp_release` restore `getpid` inside the filtered process?

Save results before removing this lab's resources and files: `./lab.sh 09 reset`

Lab 10 - Ownership, disruption budgets and Service routing

What do ownership, disruption budgets and Service routing each guarantee during Pod replacement?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 10 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A Pod groups containers and has its own object identity (UID). A Deployment expresses the desired replica count; its ReplicaSet creates Pods to maintain that count. Deleting one managed Pod leaves the desired count unchanged, so reconciliation creates a different Pod rather than resurrecting the deleted object.

The controller creates the replacement, the scheduler chooses a node, and kubelet starts its containers. These are asynchronous steps. The old Pod may still be terminating while the new one starts; a temporarily larger object count is not itself a failure.

A Service selects Pods by labels and routes through eligible endpoints. Readiness determines whether a Pod is eligible; Running alone is insufficient. Here readiness opens TCP port 8080, whereas the client asks `/healthz` for an HTTP result. Endpoint changes and existing connections can lag behind Pod changes.

Measure both recovery of replica count and sampled client availability. With 100 probes, sampled availability is successful HTTP responses / 100. Zero failed probes means none of these probes saw failure; requests between samples or other paths remain untested. Goldpinger's local `/healthz` response is not a measurement of every peer connection.

Follow `ownerReferences` from Pod to ReplicaSet to Deployment. Labels select resources; ownership identifies the controller responsible for replacement. Kubernetes does not recreate an unmanaged Pod merely because a Service selects it.

A PodDisruptionBudget (PDB) constrains voluntary evictions through the Eviction API. With three healthy selected Pods and `minAvailable=3`, `disruptionsAllowed` is zero. An eviction should be refused with a disruption-budget error; Forbidden would instead mean an authorization failure. Direct Pod DELETE bypasses this check, and a PDB cannot prevent node failures.

After replacement, deliberately change only the Service selector to match no Pods. The Pods can stay Ready while the Service loses eligible endpoints. Restoring the selector repairs discovery without restarting the application. EndpointSlices report addresses and conditions, not a per-request success guarantee.

A failure domain is a set of replicas that one failure can remove together. Replicas on the same node share node failure; separate kind nodes still share the host VM. Replica count alone does not establish resilience to either shared failure.

Source: docs/chaos-theory.md: §§10.4, 10.5.1-10.5.2; Experiment Card 10.1.

The experiment: The chaos cluster has three Goldpinger replicas. The readiness probe checks TCP 8080; HTTP probes sample /healthz through the Service.

How to read the evidence

Signal	Meaning
Pod name / UID	Compare before and after deletion to identify the replacement. A changed process restart count inside the same Pod would be a different event.
READY / endpointslices	READY reports the configured readiness result. EndpointSlice addresses and readiness show the backend set used for routing.
probes.log	Each row contains a timestamp and HTTP code. Count non-200 rows only after all 100 samples finish.
k helper	Runs kubectl against kind-lab10 in namespace chaos-labs. Node queries are cluster-scoped even though the helper specifies a namespace.
ownerReferences	Read the Pod owner, then that ReplicaSet's owner. A selected Pod and an owned Pod are different relationships.
disruptionsAllowed / eviction error	Wait for the PDB controller to report currentHealthy=3 and disruptionsAllowed=0. Only a budget rejection tests the intended guard.
Service selector / ready endpoints	An extra unmatched selector label makes the selector's AND expression false. Count endpoints with ready=true, including an empty set.

Predict

Predict whether the PDB blocks eviction and direct deletion separately. Then predict Pod readiness and HTTP results when only the Service selector is wrong.

Record your results

Phase	Identity / Ready Pods	PDB / ready endpoints	HTTP evidence
Baseline ownership			
Eviction request			
Direct deletion (100 probes)			
Wrong selector			
Selector restored			

Procedure

1. Checkpoint the prepared chaos cluster

VM terminal 1

```
source ~/labs/lab10/env.sh
k get nodes
k get pods -l app=goldpinger -o wide
k auth can-i list pods --as=system:serviceaccount:chaos-labs:goldpinger
k auth can-i delete pods --as=system:serviceaccount:chaos-labs:goldpinger
NODE_IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab10-control-plane)
curl -fsS --max-time 3 "http://$NODE_IP:30080/healthz"; echo
k get endpointslices -l kubernetes.io/service-name=goldpinger -o wide
k get pods -l app=goldpinger -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

2. Trace ownership and test a voluntary eviction guard

VM terminal 1, same shell

```
victim=$(k get pods -l app=goldpinger -o jsonpath='{.items[0].metadata.name}')
rs=$(k get pod "$victim" -o jsonpath='{.metadata.ownerReferences[0].name}')
k get pod "$victim" -o jsonpath='{.metadata.uid}{" owner="}{.metadata.ownerReferences}{"\n"}'
k get rs "$rs" -o jsonpath='{.metadata.ownerReferences}{"\n"}'
k create pdb lab10-budget --selector=app=goldpinger --min-available=3 --dry-run=client -o yaml | k apply -f -
k wait pdb/lab10-budget --for=jsonpath='{.status.currentHealthy}'=3 --timeout=60s
k wait pdb/lab10-budget --for=jsonpath='{.status.disruptionsAllowed}'=0 --timeout=60s
k get pdb lab10-budget
printf '{"apiVersion":"policy/v1","kind":"Eviction","metadata":{"name":"%s","namespace":"chaos-labs"}}\n'
"$victim" > ~/labs/lab10/eviction.json
k create --raw "/api/v1/namespaces/chaos-labs/pods/$victim/eviction" -f ~/labs/lab10/eviction.json
k get pod "$victim" -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

3. Start 100 probes about 0.2 s apart (start this right before the deletion)

VM terminal 2

```
source ~/labs/lab10/env.sh
NODE_IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab10-control-plane)
for i in $(seq 1 100); do
    code=$(curl -s --max-time 1 -o /dev/null -w '%{http_code}' "http://$NODE_IP:30080/healthz")
    printf '%s %s\n' "$(date --iso-8601=ns)" "$code"
    sleep 0.2
done | tee ~/labs/lab10/probes.log
```

4. Directly delete the same Pod despite the PDB; observe replacement for up to 60 seconds

VM terminal 1, same shell

```
echo "victim=$victim deleted_at=$(date --iso-8601=ns)"
k delete pod "$victim" --wait=false
timeout 60s kubectl --kubeconfig="$HOME/labs/lab10/kubeconfig" --context=kind-lab10 --namespace=chaos-labs get
  pods -l app=goldpinger -w
# timeout exit 124 ends observation, not the Kubernetes experiment.
```

5. After all 100 probes finish in terminal 2, collect recovery evidence

VM terminal 1, same shell

```
k rollout status deployment/goldpinger --request-timeout=0 --timeout=180s
k get pods -l app=goldpinger -o
  custom-columns='NAME:.metadata.name,READY:.status.conditions[?(@.type=="Ready")].status,START:.status.startT
ime,NODE:.spec.nodeName'
k get pod "$victim" 2>&1 | tail -1
k get endpointslices -l kubernetes.io/service-name=goldpinger -o wide
samples=$(wc -l < ~/labs/lab10/probes.log)
if [ "$samples" -eq 100 ]; then
    echo "non-200 samples: $(grep -vc ' 200$' ~/labs/lab10/probes.log)"
    grep -v ' 200$' ~/labs/lab10/probes.log | head
else
    echo "Only $samples/100 samples recorded; wait for terminal 2 to finish before counting."
fi
k get pods -l app=goldpinger -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
```

6. Break only Service selection and compare all three signals

VM terminal 1, same shell

```
k delete pdb lab10-budget --ignore-not-found
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":"lab10-no-match"}}}'
sleep 5
k get pods -l app=goldpinger
k get svc goldpinger -o jsonpath='{.spec.selector}{"\n"}'
k get endpointslices -l kubernetes.io/service-name=goldpinger -o json | jq '[.items[].endpoints[? |
  select(.conditions.ready == true)] | length'
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$NODE_IP:30080/healthz"
```

7. Restore the selector and confirm routing recovery

VM terminal 1, same shell

```
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":null}}}'
for i in $(seq 1 15); do
  curl -fs --max-time 2 "http://$NODE_IP:30080/healthz" && break
  sleep 1
done
k get endpointslices -l kubernetes.io/service-name=goldpinger -o json | jq '[.items[].endpoints[]? |
  select(.conditions.ready == true)] | length'
curl -fsS --max-time 3 "http://$NODE_IP:30080/healthz"; echo
```

Recovery check: Three replicas and three eligible Service endpoints are restored, HTTP succeeds, and lab10-budget is absent.

Only if normal recovery fails

1. Restore Service selection and remove the experiment PDB

VM terminal 1, same shell

```
source ~/labs/lab10/env.sh
k patch svc goldpinger --type merge -p '{"spec":{"selector":{"experiment":null}}}'
k delete pdb lab10-budget --ignore-not-found
```

Answer

Trace Pod -> ReplicaSet -> Deployment. Compare eviction rejection with direct deletion, replacement UID and failed samples out of 100. Explain the wrong-selector result using Pod readiness and EndpointSlice evidence, then show recovery.

Check your understanding

Would three replicas on the same node establish resilience to losing that node?

Save results before removing this lab's resources and files: `./lab.sh 10 reset`

Lab 11 - Health depends on what you test

How do request budgets and readiness probe choice change the verdict on the same slow dependency?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 11 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

Goldpinger discovers labelled peer Pods and periodically asks each for an HTTP response. A peer is healthy to a caller only if the operation succeeds within that caller's timeout. A slow but running process can fail that test.

The added Pod contains Goldpinger on port 9090 and Toxiproxy on port 8080. Containers in the Pod share a network namespace, so the proxy forwards to 127.0.0.1:9090. Peers still call port 8080 and traverse the proxy; this leaves the original replicas unchanged. Its app label matches the Service, but it lacks `component=baseline`, so the baseline ReplicaSet does not own or adopt it.

A latency toxic delays traffic in a selected direction. Here it delays upstream application data travelling from the caller through the proxy to Goldpinger. A TCP connection to the proxy can be accepted before that data is forwarded. Kubernetes' TCP readiness probe therefore tests a weaker property than the peer's timed HTTP request.

Compare no delay, 100 ms, and 400 ms against the 300 ms peer budget. Under-budget delay can show up in response time without changing a boolean health result. Over-budget delay can change peer health

while TCP readiness remains true. Reports are periodically refreshed, so wait for fresh samples and remove the toxic to test recovery.

A second run changes readiness from TCP to HTTP GET /healthz through the same proxy, with timeoutSeconds=1, periodSeconds=2 and failureThreshold=2. At 400 ms the peer’s 300 ms request can fail while the one-second readiness request succeeds. At 1400 ms both budgets are exceeded; readiness needs consecutive failed samples before changing.

Readiness failure makes this Pod ineligible for ordinary Service traffic; it does not restart a container. A liveness failure could restart it, and a startup probe would defer readiness and liveness until startup succeeds. This fixture intentionally has neither liveness nor startup probes. Record UID and restart counts to distinguish readiness recovery from replacement. Restarting a caller does not remove delay in its dependency. A liveness check tied to that dependency can repeatedly restart otherwise functioning callers and reduce available capacity.

Probe configuration on this standalone Pod cannot be changed in place. Recreate it between the TCP and HTTP comparisons, establish a fresh baseline and record its new UID. Within each comparison, changing only the toxic must not require recreation. Goldpinger discovers labelled Pod IPs directly, so removing an unready Pod from Service routing does not necessarily stop peer pings to it.

Source: docs/chaos-theory.md: §§10.4.4, 10.5.3-10.5.4; Experiment Card 10.2.

The experiment: Use this lab's three Goldpinger replicas. Compare 0, 100 and 400 ms using TCP readiness, then recreate only the extra Pod with one-second HTTP readiness and compare 0, 400 and 1400 ms. The peer budget remains 300 ms throughout.

How to read the evidence

Signal	Meaning
OK / ms / error / PingTime	The selected peer report says whether its HTTP ping met the budget, its recorded latency and any error. Compare PingTime with the printed observation time for freshness. An absent report is missing evidence, not a failed ping.
Ready	In the TCP comparison it tests only the listener. In the HTTP comparison it tests /healthz through the proxy within one second, with two consecutive failures required to become unready.
Toxiproxy toxic	The API shows the configured direction and dose. Check it before interpreting peer observations.
Repeated samples	Read each original replica directly. That avoids accidentally sampling only one backend through the Service.
HTTP readiness / EndpointSlice ready	Read the slow Pod’s own EndpointSlice condition. Other ready replicas can keep the Service available and hide this Pod’s failure.
UID / restartCount	A new UID between configurations is deliberate recreation. Within one configuration, unchanged UID and restart counts support readiness-only recovery.

Predict

Compare the 300 ms peer budget with TCP readiness and one-second HTTP readiness at 400 ms and 1400 ms. Predict endpoint eligibility and restart counts separately.

Record your results

Probe / dose	Peer health / latency	Pod Ready / endpoint ready	UID / restarts
TCP / 0 ms			
TCP / 100 ms			
TCP / 400 ms			
HTTP / 0 ms			
HTTP / 400 ms			
HTTP / 1400 ms			
HTTP / removed			

Procedure

1. Connect the prepared proxy and establish the no-delay baseline

VM terminal 1

```
source ~/labs/lab11/connect-proxy.sh
pings
readiness
```

2. Add 100 ms, below the peer budget

VM terminal 1

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics" -d '{"name":"lat","type":"latency","stream":"upstream","toxicity":1,"attributes":{"latency":100,"jitter":0}}'
sleep 8
pings
readiness
```

3. Change the same toxic to 400 ms, above the budget

VM terminal 1

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics/lat" -d '{"attributes":{"latency":400,"jitter":0}}'
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
sleep 8
pings
readiness
```

4. Recreate only the experimental Pod with HTTP readiness

VM terminal 1, same shell

```
curl -fsS --max-time 5 -X DELETE "$T/proxies/slow/toxics/lat"
stop_forward
k delete pod goldpinger-slow --wait=true --timeout=60s
source ~/labs/lab11/connect-proxy.sh ~/labs/lab11/slow-http.yaml
pings
readiness
```

5. Compare 400 ms with the longer readiness budget

VM terminal 1, same shell

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics" \
-d '{"name":"lat","type":"latency","stream":"upstream","toxicity":1,"attributes":{"latency":400,"jitter":0}}'
sleep 10
pings
readiness
```

6. Exceed both budgets and watch eligibility without restarting

VM terminal 1, same shell

```
curl -fsS --max-time 5 -H 'Content-Type: application/json' -X POST "$T/proxies/slow/toxics/lat" \
-d '{"attributes":{"latency":1400,"jitter":0}}'
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
for i in $(seq 1 6); do date --iso-8601=seconds; readiness; sleep 3; done
pings
```

7. Remove the fault, verify recovery, delete the pod and stop the port-forward

VM terminal 1, same shell

```
curl -fsS --max-time 5 -X DELETE "$T/proxies/slow/toxics/lat"
curl -fsS --max-time 5 "$T/proxies/slow/toxics"; echo
k wait --for=condition=Ready pod/goldpinger-slow --timeout=60s
sleep 8
pings
readiness
stop_forward
k delete pod goldpinger-slow --wait=true --timeout=60s
k rollout status deployment/goldpinger --timeout=180s
k get pods -l app=goldpinger
```

Recovery check: The toxic is removed and the original three replicas are Ready.

Answer

Report all seven rows. Explain the 400 ms disagreement under both probe types, the readiness transition at 1400 ms, and why direct peer discovery may still find an unready Pod. Show readiness recovery without a restart within the HTTP comparison.

Check your understanding

Would adding liveness with the same slow dependency necessarily improve availability?

Save results before removing this lab's resources and files: `./lab.sh 11 reset`

Lab 12 - Locate startup failures and measure recovery

Can the startup SLI detect different failure stages, distinguish their causes and measure recovery?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 12 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

An SLI is a measured service outcome; an SLO is a target for it. This lab measures elapsed time from submitting a fresh Pod and Service to receiving HTTP 200 through that Service. The per-trial target is at most 30 seconds. Pod Running is an earlier, weaker milestone.

Startup includes API submission, scheduling, container startup, readiness and usable routing. The image is cached, so this experiment does not include a cold image download. Repeating a warm-start check cannot establish a cold-start guarantee. With `imagePullPolicy=Always`, the runtime checks the registry to resolve the image to a digest (a content identifier), but can reuse cached layers. Always does not mean every layer is downloaded again; registry access and layer transfer are different startup costs.

Test the checker as well as the workload: a known 45-second startup delay should exceed its 30-second budget, and restoring the normal command should recover. Delete the previous Pod and Service before each trial so an old server cannot answer for the new one.

A completed trial can report success=0; that is valid evidence of a missed target. A checker error or stale metrics is not a completed failed trial. The last-run timestamp identifies fresh results. Over many representative trials the fraction meeting the target forms an SLI; these diagnostic trials do not establish a long-term SLO.

The same HTTP deadline miss can have different causes. An unmatched required nodeSelector leaves a Pod unscheduled: PodScheduled=False, reason Unschedulable, and no nodeName. A delayed application has a node but cannot yet pass readiness. A wrong Service selector leaves a healthy Ready Pod with zero matching endpoints. Use these intermediate observations to locate the failed stage; success=0 alone does not identify it.

A container restart reruns the same configured command. It does not change nodeSelector or Service labels, and a command containing a 45-second sleep repeats that sleep after restart. Recovery must change the setting responsible for the failed stage.

The checker saves a per-trial JSON snapshot of the Pod, Service, EndpointSlices and Pod events before cleanup. Capture occurs after the timed HTTP measurement, so it does not increase the measured startup duration. Each read is a separate observation, not an atomic cluster snapshot; diagnostics errors are recorded explicitly.

Run three healthy trials after removing the faults. The fraction meeting the target is successful completed trials / all completed trials. Keep deliberate fault trials separate from the healthy sample. A small controlled sample tests repeatability, not a production reliability target; a missed trial has no measured eventual time-to-success.

Source: docs/chaos-theory.md: §§11.2, 11.4.1-11.4.3; scheduling in §12.1.1.

The experiment: Setup prepares the checker and manifest. Each invocation creates a fresh workload, waits up to its 30-second budget, records a result and cleans up. The image is preloaded.

How to read the evidence

Signal	Meaning
success / elapsed	success=1 means an HTTP 200 was observed within the budget. success=0 means none was observed by the deadline. elapsed is seconds from submission.
checker exit	0 means the checker completed and recorded trials, including misses. Nonzero means execution or cleanup failed; inspect the output before using metrics.
ce_last_run_timestamp_seconds	Unix timestamp of the last completed trial. It must advance; old output cannot represent a new run.
NotFound after cleanup	Both Pod and Service should be absent. An API connection error is not evidence that they were deleted.
PodScheduled / nodeName / Ready	No node and Unschedulable locates placement failure. Assigned but unready locates a later stage. Ready alone does not prove Service routing.
diagnostics/run-N.json	Contains the trial timestamp, measurement and pre-cleanup API evidence. Errors mean the corresponding diagnostic was unavailable; they are not an empty successful query.
Healthy sample SLI	Calculate the fraction from the three run output lines, not start.prom, which contains only the most recent trial.

Predict

Predict success and the distinguishing Pod/endpoint evidence for delayed startup, impossible placement and wrong Service selection. Predict which faults restarting the container could repair.

Record your results

Trial	success / elapsed	Scheduled / Ready	Eligible endpoints / cause
Normal			
45 s delay			
Unmatched nodeSelector			
Wrong Service selector			
Restored trial 1			
Restored trial 2			
Restored trial 3			

Procedure

1. Run one healthy check and confirm cleanup

VM terminal 1

```
cd ~/labs/lab12
source ~/labs/lab12/env.sh
bash run.sh | tee runs.txt
echo "checker exit=${PIPESTATUS[0]} (0 means the checker and cleanup completed)"
cat metrics/start.prom
k get pod startup-check; k get svc startup-check
cp diagnostics/run-1.json evidence-normal.json
```

2. Prove the check detects a deliberately delayed startup

VM terminal 1

```
cd ~/labs/lab12
cp startup-delayed.yaml startup.yaml
grep -n 'command:' startup.yaml
bash run.sh | tee runs-delayed.txt
echo "checker exit=${PIPESTATUS[0]} (0 means the checker and cleanup completed)"
cp diagnostics/run-1.json evidence-delayed.json
jq '{trial, conditions: .pod.status.conditions, containers: .pod.status.containerStatuses}'
evidence-delayed.json
```

3. Make placement impossible without changing the application

VM terminal 1, same shell

```
cp startup-unscheduled.yaml startup.yaml
bash run.sh | tee runs-unscheduled.txt
echo "checker exit=${PIPESTATUS[0]}"
cp diagnostics/run-1.json evidence-unscheduled.json
jq '{trial, pod: .pod.status, spec: .pod.spec.nodeSelector, events: .events}' evidence-unscheduled.json
```

4. Break Service matching while allowing the Pod to become Ready

VM terminal 1, same shell

```
cp startup-selector.yaml startup.yaml
bash run.sh | tee runs-selector.txt
echo "checker exit=${PIPESTATUS[0]}"
cp diagnostics/run-1.json evidence-selector.json
jq '{trial, conditions: .pod.status.conditions, selector: .service.spec.selector, slices: .endpointslices}'
evidence-selector.json
```

5. Restore the healthy manifest, run three trials and calculate the sample SLI

VM terminal 1

```
cd ~/labs/lab12
cp startup-good.yaml startup.yaml
grep -n 'command:' startup.yaml
bash run.sh 3 | tee runs-restored.txt
echo "checker exit=${PIPESTATUS[0]} (0 means the checker and cleanup completed)"
cat metrics/start.prom
k get pod startup-check; k get svc startup-check

mkdir -p evidence-restored
cp diagnostics/run-*.json evidence-restored/
awk '/^run=/ {n++; if ($2 == "success=1") good++;} END {if (n == 3) printf "healthy sample: %d/%d = %.3f\n",
    good,n,good/n; else print "Incomplete sample; inspect checker exit"}' runs-restored.txt
```

Recovery check: The healthy manifest is restored and no startup-check Pod or Service remains.

Only if normal recovery fails

1. Manual fallback if the loop was killed

VM terminal 2

```
source ~/labs/lab12/env.sh
k delete -f ~/labs/lab12/startup.yaml --ignore-not-found --grace-period=1
```

Answer

Complete the seven rows, explain the earliest failed stage in each fault, and calculate the restored sample's success fraction. Show fresh diagnostics and cleanup; distinguish a completed miss, missing diagnostics and a checker execution error.

Check your understanding

Does imagePullPolicy: Always force every image layer to download on each run?

Save results before removing this lab's resources and files: `./lab.sh 12 reset`

Lab 13 - Placement, supervision and replacement

How do cordoning and stopping kubelet differ in scheduling, existing execution and replacement?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 13 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

Kubelet is the node agent that reports state and asks the container runtime to start or stop containers. The runtime executes them separately. Stopping kubelet removes supervision and reporting without inherently stopping an already-running application.

Missing reports must first be detected. The control plane then marks the node unavailable and applies a NoExecute taint. A Pod's tolerationSeconds says how long it tolerates that taint before eviction begins; it is not measured from the moment kubelet stopped.

This lab explicitly gives node-web 20-second not-ready and unreachable tolerations to shorten the comparison. Total replacement time still includes node detection, eviction processing, scheduling and startup. Do not infer a universal Kubernetes recovery time from this configured example.

A replacement Pod can be created elsewhere while the original process remains on the unsupervised node. The API records desired and reported state; it cannot by itself prove that an unreachable node executed a stop. Compare the original container ID in `crictl` with the new Pod UID, then restore kubelet so

the old node can reconcile.

Cordon marks a node unschedulable for ordinary new Pods; it does not evict or stop existing Pods and does not turn off kubelet. A replacement created after you delete a managed Pod must find another eligible node. Uncordon permits future placement again; it does not move existing Pods back.

The node-web workload prefers lab13-worker but does not require it. A preference can be overridden when that node is unavailable; a required hostname constraint could instead leave a replacement Pending. Record placement rather than assuming that replica count guarantees independent failure domains. All kind nodes still share this VM.

Record the Lease renewTime separately from the Node Ready condition. A stale Lease shows missing heartbeats; later conditions and NoExecute taints drive a different stage. Timestamped samples place events between observations; ten-second sampling does not measure exact detection latency.

Source: docs/chaos-theory.md: §§12.1.1-12.1.3 and 12.3.1-12.3.2.

The experiment: First cordon the worker hosting node-web and test replacement. Restore scheduling, then stop kubelet on the replacement's worker. Sample 18 times, sleeping ten seconds between samples; API calls add time. Restore kubelet even if no replacement was observed.

How to read the evidence

Signal	Meaning
Node Ready / taints	A missing heartbeat can produce Ready=Unknown, displayed as NotReady. Record when the taint appears separately from the stop time.
Pod UID / node	A new UID on another node is a replacement. The old object may remain Terminating until its kubelet returns.
crictl ps	Queries the runtime inside the original kind node directly, independent of kubelet's API reports. Match the recorded original container ID.
20-second toleration	Starts when the matching NoExecute taint applies. It neither stops the old process nor guarantees replacement exactly 20 seconds later.
spec.unschedulable	true blocks ordinary new placement but says nothing about the existing process or node health.
Lease renewTime	Node heartbeats are represented by a Lease in kube-node-lease. Compare renewTime before and during loss of kubelet supervision.
Observed time intervals	Compute stop -> first unavailable sample and first taint -> first replacement sample separately; mark unresolved events as not observed.

Predict

Predict which operation changes scheduling eligibility, which stops reporting, and whether either immediately kills existing containers. Predict whether uncordon moves the replacement back.

Record your results

Event	Time	Node / Pod API state	Original container running?
Baseline			
Cordoned (same UID)			
Deleted while cordoned			
Uncordoned (no automatic move)			
Kubelet stopped			
Lease stale / taint observed			
Replacement observed			
Kubelet restored			

Procedure

1. Record the original Pod and runtime container

VM terminal 1

```
source ~/labs/lab13/env.sh
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NODE=$(k get pod "$POD" -o jsonpath='{.spec.nodeName}')
echo "original pod=$POD node=$NODE"
k get pod "$POD" -o jsonpath='{.metadata.uid}{"\n"}{.spec.tolerations}{"\n"}'
docker exec "$NODE" crictl ps --name web
```

2. Cordon the host and prove the existing Pod stays in place

VM terminal 1, same shell

```
ORIGINAL_NODE=$NODE
k cordon "$ORIGINAL_NODE"
k get node "$ORIGINAL_NODE" -o jsonpath='{.spec.unschedulable}{"\n"}'
k get pod "$POD" -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
docker exec "$ORIGINAL_NODE" systemctl is-active kubelet
docker exec "$ORIGINAL_NODE" crictl ps --name web
```

3. Delete while cordoned; locate the replacement, then uncordon

VM terminal 1, same shell

```
k delete pod "$POD" --wait=true --timeout=60s
k rollout status deployment/node-web --timeout=180s
k get pods -l app=node-web -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
k uncordon "$ORIGINAL_NODE"
sleep 5
k get pods -l app=node-web -o custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName'
POD=$(k get pods -l app=node-web -o jsonpath='{.items[0].metadata.name}')
NODE=$(k get pod "$POD" -o jsonpath='{.spec.nodeName}')
OLD_CID=$(docker exec "$NODE" crictl ps --name web -q)
echo "kubelet trial: pod=$POD node=$NODE container=$OLD_CID"
k get pod "$POD" -o jsonpath='{.metadata.uid}{"\n"}'
kubectrl --kubeconfig="$HOME/labs/lab13/kubeconfig" -n kube-node-lease get lease "$NODE" -o
  jsonpath='{.spec.renewTime}{"\n"}'
```

4. Stop kubelet and collect 18 timestamped samples, roughly ten seconds apart

VM terminal 1

```
date --iso-8601=seconds
docker exec "$NODE" systemctl stop kubelet
for i in $(seq 1 18); do
  date --iso-8601=seconds
  k get node "$NODE"
  k get node "$NODE" -o jsonpath='{.spec.taints}{"\n"}'
  k get pods -l app=node-web -o
    custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName,DELETING:.metadata.deletionTimestamp'
  kubectl --kubeconfig="$HOME/labs/lab13/kubeconfig" -n kube-node-lease --request-timeout=5s get lease "$NODE"
    -o jsonpath='{.spec.renewTime}{"\n"}'
  echo "original runtime ID=$OLD_CID"
  docker exec "$NODE" crictl ps --name web
  sleep 10
done | tee ~/labs/lab13/timeline.txt
```

5. Restore kubelet and inspect convergence

VM terminal 1

```
docker exec "$NODE" systemctl start kubelet
k wait --for=condition=Ready "node/$NODE" --request-timeout=0 --timeout=180s
k rollout status deployment/node-web --request-timeout=0 --timeout=180s
k get nodes
k get pods -l app=node-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,NODE:.spec.nodeName,DELETING:.metadata.deletionTimestamp'
docker exec "$NODE" crictl ps --name web
k get nodes -o custom-columns=NAME:.metadata.name,UNSCHEDULABLE:.spec.unschedulable
echo "original runtime ID=$OLD_CID"
```

Recovery check: Kubelet is running on every worker, four nodes are Ready and uncordoned, and one node-web Pod is Ready. If replacement occurred, confirm the recorded old container is gone.

Only if normal recovery fails

1. Restore kubelet even if the Kubernetes API is unavailable

VM terminal 2

```
for node in lab13-worker lab13-worker2 lab13-worker3; do
  docker exec "$node" systemctl start kubelet
done

# After the API responds, restore placement eligibility too.
source ~/labs/lab13/env.sh
k uncordon lab13-worker lab13-worker2 lab13-worker3
```

Answer

Compare UID and node across cordon, deletion and uncordon. Then build a separate kubelet-failure timeline using Lease, taint, Pod UID and original runtime ID. Explain detection, eviction, replacement and cleanup separately; report unobserved transitions without inventing times.

Check your understanding

Why would deleting the old Pod object forcibly be insufficient to prove there is only one running copy?

Save results before removing this lab's resources and files: `./lab.sh 13 reset`

Lab 14 - Quorum, convergence and application availability

How does quorum loss affect committed configuration, Deployment convergence and existing HTTP traffic?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 14 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

etcd stores Kubernetes state and uses Raft consensus. A change needs a majority of the configured voting members: $\text{floor}(n/2) + 1$. For three members that is two. Stopping a member does not remove it from membership or lower the required majority. floor rounds down; failure tolerance is n minus the required majority, assuming the survivors can communicate.

A leader coordinates committed changes. Losing one member can cause a transition or election but leaves a possible majority. Losing two prevents new commits. A minority cannot accept independent changes without risking conflicting histories.

The management path is `kubectl` -> API server -> etcd. The existing application path is HTTP client -> NodePort routing -> already-running web Pod. That routing need not perform an etcd write for every request, so these two paths can have different availability during quorum loss.

Test a real ConfigMap change, not only a read that might be cached, and separately probe HTTP. Move a static etcd manifest out of kubelet's watched directory to stop that member. Restore the same manifests through Docker, a path independent of the Kubernetes API. This recovers temporarily stopped members with intact data; it is not a backup-restore experiment.

A successful scale request commits a desired replica count; it does not mean those replicas are already Ready. Compare `spec.replicas` with `status.readyReplicas` and wait for rollout completion. This separates the storage operation from controller, scheduler and kubelet convergence.

A client timeout is an unknown result: the caller lacks an acknowledgement and must not infer that no change committed. After recovery, read the ConfigMap value and Deployment spec before overwriting them. Restore an idempotent target, meaning it is safe to repeat: set replicas to two rather than incrementing on each retry. Verify the observed result. A successful GET during disruption can be cached or use a different path and is weaker evidence than a new committed write.

Record etcd member identities and leader before and after losing one member. This lab stops a fixed member; a leader change is only expected if the stopped member was leader or another election occurred. Do not label every single-member failure a leader-election experiment. Two surviving voters suffice regardless of which member was previously leader.

Source: docs/chaos-theory.md: §§12.1.1, 12.1.4 and 12.3.3-12.3.4.

The experiment: Use Lab 14's own lab14 cluster. Move static etcd manifests to stop members and restore the same manifests to recover them; Docker provides a fallback independent of `kubectl`. Setup points control-plane kubelets at the HA API load balancer, so restoring a local etcd member is not tied to that member's unavailable API server.

How to read the evidence

Signal	Meaning
endpoint status / health	Status identifies members and leader. Health checks test whether the endpoint can participate successfully; read all three after restoration.
crictl ps --name etcd	Confirms the selected member actually stopped. Moving the file alone is only the injection request.
ConfigMap patch	Change the state value each phase. A successful committed change exercises the write path; a no-op or cached get is weaker evidence.
HTTP code	200 from the recorded worker NodePort measures the existing data path, independently of kubectl.
spec.replicas / readyReplicas	Desired replica count is committed configuration; Ready count is asynchronous workload convergence. A scale command alone measures only the former.
ConfigMap value / resourceVersion	Read the value after recovery to resolve an uncertain write. resourceVersion is an opaque version identifier, not a number for timing or arithmetic.
Member ID / leader / raft term	Identify whether the chosen stopped member was leader before attributing an election to that fault.

Predict

Calculate quorum for 3, 4 and 5 voting members. Predict ConfigMap writes, scaling convergence and HTTP for one and two stopped members; predict whether a leader change is required for the fixed first target.

Record your results

Phase	Write / stored value	Desired / Ready replicas	HTTP / leader evidence
3 voters / baseline 2 replicas			
2 voters / scale to 3			
1 voter / attempted scale to 1			
Restored before overwriting			
Restored target 2 replicas			

Procedure

1. Establish HTTP and write baselines

VM terminal 1

```
source ~/labs/lab14/helpers.sh
h get nodes
h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
PORT=$(h get svc quorum-web -o jsonpath='{.spec.ports[0].nodePort}')
IP=$(docker inspect -f '{{.NetworkSettings.Networks.kind.IPAddress}}' lab14-worker)
curl -fsS --max-time 3 "http://$IP:$PORT/" >/dev/null && echo 'HTTP baseline ok'
h patch configmap quorum-probe --type merge -p '{"data":{"state":"baseline"}}'
ec lab14-control-plane endpoint status --cluster -w table
ec lab14-control-plane member list -w table
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
```

2. Stop one etcd member and probe both paths

VM terminal 1, same shell

```
stop_etcd lab14-control-plane2
docker exec lab14-control-plane2 crictl ps --name etcd
h patch configmap quorum-probe --type merge -p '{"data":{"state":"one-down"}}'
curl -sS --max-time 3 -o /dev/null -w '%{http_code}\n' "http://$IP:$PORT/"

ec lab14-control-plane endpoint status -w table
ec lab14-control-plane3 endpoint status -w table
h scale deployment/quorum-web --replicas=3
h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
```

3. Lose quorum and attempt both configuration and scaling writes

VM terminal 1, same shell

```
stop_etcd lab14-control-plane3
docker exec lab14-control-plane3 crictl ps --name etcd
h patch configmap quorum-probe --type merge -p '{"data":{"state":"two-down"}}'
echo "ConfigMap write exit=$?"
h scale deployment/quorum-web --replicas=1
echo "scale exit=$?"
h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
curl -sS --max-time 3 -o /dev/null -w 'HTTP=%{http_code}\n' "http://$IP:$PORT/"
```

4. Restore members, then resolve uncertain operations before changing state

VM terminal 1, same shell

```
start_etcd lab14-control-plane2
start_etcd lab14-control-plane3
healthy=0
for i in $(seq 1 18); do
  if ec lab14-control-plane endpoint health --cluster; then healthy=1; break; fi
  sleep 5
done
test "$healthy" -eq 1 || echo 'STOP: use fallback; do not claim quorum recovery.'
observed=0
for i in $(seq 1 18); do
  if h get configmap quorum-probe -o jsonpath='{.data.state}{.resourceVersion}{.metadata.resourceVersion}{"\n"}' &&
    h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas; then
    observed=1; break
  fi
  sleep 5
done
test "$observed" -eq 1 || echo 'STOP: retain the unknown result; do not overwrite it before readback works.'
ec lab14-control-plane endpoint status --cluster -w table
```

5. Restore the explicit baseline and verify convergence and HTTP independently

VM terminal 1, same shell

```
if [[ ${healthy:-0} -eq 1 && ${observed:-0} -eq 1 ]]; then
  h patch configmap quorum-probe --type merge -p '{"data":{"state":"recovered"}}'
  h scale deployment/quorum-web --replicas=2
  h rollout status deployment/quorum-web --request-timeout=0 --timeout=180s
  h get configmap quorum-probe -o jsonpath='{.data.state}{"\n"}'
  h get deployment quorum-web -o custom-columns=DESIRED:.spec.replicas,READY:.status.readyReplicas
  ec lab14-control-plane endpoint health --cluster
  curl -fsS --max-time 3 "http://$IP:$PORT/" >/dev/null && echo 'HTTP recovery ok'
else
  echo 'STOP: complete quorum recovery and readback in the previous step before overwriting state.'
fi
```

Recovery check: All three etcd endpoints are healthy, the recovered ConfigMap value is readable, desired and Ready replicas are two, and HTTP succeeds.

Only if normal recovery fails

1. Independent fallback that restores the manifests without kubectl

VM terminal 2

```
for node in lab14-control-plane2 lab14-control-plane3; do
  docker exec "$node" sh -c 'test -f /root/ce-etcd.yaml && mv /root/ce-etcd.yaml
    /etc/kubernetes/manifests/etcd.yaml; ls /etc/kubernetes/manifests'
done
```

Answer

Compare all five phases. Separate write acceptance, Ready replica convergence and existing HTTP. Resolve timed-out writes by reading stored values after recovery, explain any leader change from member evidence, and calculate failure tolerance for 3, 4 and 5 voters.

Check your understanding

Would four configured members tolerate two failures?

Save results before removing this lab's resources and files: `./lab.sh 14 reset`

Lab 15 - Count how layered retries multiply work

How much extra backend work do layered retries create when the same dependency keeps failing?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 15 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

An original user request is not the same as an attempt. One retry means up to two attempts. When a client retries a proxy call and the proxy retries each upstream call, the per-layer attempt limits multiply: at most $\text{client attempts} \times \text{proxy attempts} \times \text{backend arrivals}$ per original request.

Here HTTP 503 means Service Unavailable. NGINX can make two upstream attempts for that response, and the client makes either one or two proxy attempts. Both upstreams deliberately return 503 throughout the fault phase. A successful attempt stops retries early, so the maximum is reached only when the earlier attempts continue failing.

Measure amplification as $\text{backend arrivals} / \text{original requests}$, and useful work as $\text{successful original requests} / \text{original requests}$. More attempts do not imply more successes. Keep the original request count and failure condition identical when changing the client retry count.

Layered retry amplification is the first step in the book's emergent retry-storm example. This small, sequential experiment isolates multiplication; it does not simulate overloaded worker queues or prove self-sustaining failure. In a real system, retries of slow requests may add work while earlier attempts are still running; bounded attempts, a total deadline and backoff address different parts of that problem.

Source: docs/chaos-theory.md: §1.2.3 (emergent retry amplification), §12.1.4 (ingress retry accumulation). This fixture isolates attempt multiplication using HTTP 503.

The experiment: Each case sends 20 sequential requests in three phases: healthy, both upstreams returning HTTP 503, and recovered. NGINX allows two upstream attempts. Repeat with zero and one client retry. The runner owns its proxy inside `ce-lab15.service`, has a 60-second runtime limit, and removes the fault and processes on exit. Reset stops that unit and its children.

How to read the evidence

Signal	Meaning
original / client attempts	Each case has 20 original requests. Count all proxy attempts, including retries, separately.
backend arrivals	Sum the two upstream request logs within the phase. Probe and health requests are excluded.
amplification	backend arrivals / 20. Compare it with the theoretical upper bound; fewer arrivals can indicate early success or a failure before reaching a backend.
successes	Counts original requests ending in HTTP 200, not the number of attempts. Compare outcomes alongside cost.

Predict

For 20 requests, calculate the maximum backend arrivals with zero versus one client retry, given two proxy attempts. Predict success counts during a persistent 503 fault.

Record your results

Case / phase	Client attempts	Backend arrivals / amplification	Successes / 20
No client retry: baseline			
No client retry: 503 fault			
No client retry: recovered			
One client retry: baseline			
One client retry: 503 fault			
One client retry: recovered			

Procedure

1. Measure baseline, persistent failure and recovery without a client retry

VM terminal 1

```
cd ~/labs/lab15
sudo systemd-run --unit=ce-lab15 --collect --wait --pipe --uid="$(id -u)" --property=RuntimeMaxSec=60s
  --working-directory="$PWD" /bin/bash "$PWD/run-case.sh" 0 | tee no-client-retry.txt
echo "runner exit=${PIPESTATUS[0]} (0 means completed; nonzero means failure or the 60-second limit)"
```

2. Repeat the same three phases with one client retry

VM terminal 1

```
cd ~/labs/lab15
sudo systemd-run --unit=ce-lab15 --collect --wait --pipe --uid="$(id -u)" --property=RuntimeMaxSec=60s
  --working-directory="$PWD" /bin/bash "$PWD/run-case.sh" 1 | tee one-client-retry.txt
echo "runner exit=${PIPESTATUS[0]}"
```

3. Check cleanup and compare the summaries

VM terminal 1

```
cd ~/labs/lab15
test ! -e fail && echo 'fault marker removed'
ss -ltn '( sport = :8090 or sport = :9001 or sport = :9002 )'
cat no-client-retry.txt one-client-retry.txt
```

Recovery check: Each runner removes the fail marker and stops its own upstreams and proxy. Both recovered phases should return 20/20 successes.

Answer

Compare client attempts, backend arrivals / 20 and successful original requests / 20. Explain which attempt limits multiply and why restoring HTTP success stops retries early.

Check your understanding

If both layers permit three retries, what is the maximum amplification?

Save results before removing this lab's resources and files: `./lab.sh 15 reset`

Lab 16 - Design one clear experiment

How would you test one resilience claim so the result has a clear meaning?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 16 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

An experiment tests a specific service claim under a specific fault. Start with a user-visible measurement: for example, successful HTTP responses divided by total probes, or response time in milliseconds. An SLI is that measured quantity; an SLO is the target you choose for it.

Write the baseline and hypothesis using the same operation, load and measurement window. For example: "With one of three replicas unavailable for 20 seconds, at least 99 of 100 probes return HTTP 200 within one second." This states a condition and a falsifiable outcome; replace the numbers with ones justified by your system.

Specify the exact target and dose, and how to prove the fault reached it. A selector matching nothing gives a misleading pass; one matching too much changes the experiment. A control with no fault and a recovery measurement help distinguish the injected cause from an already-broken baseline.

Define a stop threshold, the command that removes the fault, and a fallback that works when the normal control path fails. Analysis must connect measurements to a mechanism and limit the conclusion to the tested conditions. This lab produces a plan, so leave observations unclaimed until an authorized run supplies them.

Source: docs/chaos-theory.md: §§1.3, 2.5; Appendix C; Master Cheat Sheet §6.

The experiment: Fill the prepared card for one service and one fault. Nothing is deployed or disrupted in this lab.

How to read the evidence

Signal	Meaning
Baseline / hypothesis	Use a named measurement with units, load and duration in both. "Healthy" without a criterion is not enough.
Fault confirmation	Name the evidence that proves the intended target and dose were affected.
Stop / recovery	Stop threshold tells the operator when to abort. Recovery criteria tell them whether service was restored; these are different checks.
Blank-field check	Finds empty fields only. A reviewer must still judge whether the plan is measurable and executable.

Predict

State what you expect to measure under one fault and why the service should behave that way.

Record your results

Design element	Your choice	How another operator checks it
Measurement / baseline		
Fault / target / duration		
Prediction / threshold		
Stop / rollback / recovery		

Procedure

1. Fill the short experiment card

VM terminal 1

```
cd ~/labs/lab16
${EDITOR:-nano} experiment-card.md
```

2. Find unanswered fields

VM terminal 1

```
cd ~/labs/lab16
grep -nE '[:space:]*$' experiment-card.md
check_status=$?
case "$check_status" in
  0) echo 'Fill the fields listed above.' ;;
  1) echo 'No blank fields found; review the answers for clarity.' ;;
  *) echo 'Could not read the card; fix the file error before continuing.' >&2 ;;
esac
```

Recovery check: The card answers each field clearly; this is a design review, not a live test.

Answer

Write a card another person can follow: measurable baseline, one fault, expected result, stop condition, rollback and recovery evidence. Keep it a plan; do not invent observations.

Check your understanding

Can a completed card or a passing no-fault control establish resilience to the proposed disruption?

Save results before removing this lab's resources and files: `./lab.sh 16 reset`

Lab 17 - Signal delivery and the shutdown budget

Why can the same application shut down cleanly in one container and be killed in another?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 17 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

Docker stop sends the configured stop signal to the container's main process, PID 1. After the stop timeout it forcibly kills a container that has not exited. Here the signal is explicitly SIGTERM; the worker handles it, spends two seconds cleaning up, writes a marker and exits zero.

A shell wrapper can sit between Docker and the application. This fixture deliberately ignores TERM in the wrapper and starts Python as a child. The child's own handler is installed, but Docker's signal does not reach it. Seeing a handler in application code is therefore insufficient; inspect the process relationship.

The shell exec builtin replaces the shell with the application without creating a child. It gives the worker PID 1 in the second configuration. Exec fixes this delivery path; it does not add a handler or make slow cleanup faster. A general-purpose init can reap orphaned children, but cannot repair an application that ignores its own termination signal.

SIGTERM (15) can be handled; SIGKILL (9) cannot be caught for cleanup. A one-second stop budget cannot accommodate this worker's two-second cleanup, even with correct delivery. Compare TERM receipt with completion. Exit 137 corresponds to $128 + 9$, consistent with SIGKILL, but does not identify its cause; check Docker's OOMKilled flag and the injected operation.

Container restart policies concern what happens after exit. They neither forward signals nor extend the shutdown budget. This experiment disables automatic restart and retains stopped containers so logs and exit evidence remain readable.

Source: docs/chaos-theory.md: §§5.13.1-5.13.2.

The experiment: Compare a non-forwarding wrapper with a five-second budget, exec with the same budget, then exec with one second. Only one comparison variable changes at a time.

How to read the evidence

Signal	Meaning
ready pid / ppid	The startup log identifies whether the worker is PID 1 or a child. Wait for this log before stopping it.
received / cleanup-complete	Receipt proves delivery to Python. The completion line and file prove this fixture finished cleanup; absence alone needs the exit and timeout evidence.
ExitCode / OOMKilled	Separate a successful handler exit from a forced stop. False OOMKilled plus the controlled stop distinguishes this kill from the memory experiment.
stop --timeout	The grace budget in seconds, not an application request timeout. The Docker command itself can succeed even when the container was killed.

Predict

Predict the worker PID, TERM receipt, cleanup completion and exit status in all three cases.

Record your results

Case	Worker PID	TERM / cleanup	Exit / OOMKilled
Wrapper / 5 s			
Exec / 5 s			
Exec / 1 s			

Procedure

1. Prepare the non-forwarding wrapper and wait for the worker

VM terminal 1

```
docker create --name ce-lab17-wrapper --stop-signal SIGTERM python:3.12-slim sh -c 'trap "" TERM; python -u /worker.py & wait'
docker cp ~/labs/lab17/worker.py ce-lab17-wrapper:/worker.py
docker start ce-lab17-wrapper
for i in $(seq 1 30); do docker logs ce-lab17-wrapper 2>&1 | grep -q '^ready ' && break; sleep 0.2; done
docker logs ce-lab17-wrapper
```

2. Stop with five seconds and preserve the evidence

VM terminal 1, same shell

```
time docker stop --timeout 5 ce-lab17-wrapper
docker logs ce-lab17-wrapper
docker inspect -f '{{json .State}}' ce-lab17-wrapper
docker cp ce-lab17-wrapper:/cleanup-complete ~/labs/lab17/wrapper-complete.txt
```

3. Change only signal delivery; keep the five-second budget

VM terminal 1, same shell

```
docker create --name ce-lab17-exec --stop-signal SIGTERM python:3.12-slim sh -c 'exec python -u /worker.py'
docker cp ~/labs/lab17/worker.py ce-lab17-exec:/worker.py
docker start ce-lab17-exec
for i in $(seq 1 30); do docker logs ce-lab17-exec 2>&1 | grep -q '^ready ' && break; sleep 0.2; done
docker logs ce-lab17-exec
time docker stop --timeout 5 ce-lab17-exec
docker logs ce-lab17-exec
docker inspect -f '{{json .State}}' ce-lab17-exec
docker cp ce-lab17-exec:/cleanup-complete ~/labs/lab17/exec-complete.txt
cat ~/labs/lab17/exec-complete.txt
```

4. Keep exec, reduce only the shutdown budget, then clean up

VM terminal 1, same shell

```
docker create --name ce-lab17-short --stop-signal SIGTERM python:3.12-slim sh -c 'exec python -u /worker.py'
docker cp ~/labs/lab17/worker.py ce-lab17-short:/worker.py
docker start ce-lab17-short
for i in $(seq 1 30); do docker logs ce-lab17-short 2>&1 | grep -q '^ready ' && break; sleep 0.2; done
docker logs ce-lab17-short
time docker stop --timeout 1 ce-lab17-short
docker logs ce-lab17-short
docker inspect -f '{{json .State}}' ce-lab17-short
docker cp ce-lab17-short:/cleanup-complete ~/labs/lab17/short-complete.txt
docker rm ce-lab17-wrapper ce-lab17-exec ce-lab17-short
```

Recovery check: The exec/five-second case completed cleanup, the two forced cases have explicit evidence, and all three test containers are removed after evidence collection.

Answer

Use the process relationship, logs, completion marker and Docker state to separate missing signal delivery from insufficient cleanup time.

Check your understanding

Would increasing the wrapper's stop budget to 30 seconds repair its missing signal forwarding?

Save results before removing this lab's resources and files: `./lab.sh 17 reset`

Lab 18 - Persistence, ownership and read-only mounts

Is a failed container write caused by lost data, Unix permissions or a read-only mount?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 18 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A container's writable layer belongs to that container. Stop and start preserve it; removing and replacing the container does not. A named volume has a separate lifetime and remains when a container using it is removed. Persistence is not a backup or a guarantee against deleting the volume.

Linux permissions compare numeric UIDs and GIDs, not account names. Here the volume directory starts owned by UID 0 with mode 0755. UID 10001 can read and traverse it but cannot create files because only the owner has directory write permission. Creating a file requires write and execute permission on its parent directory.

Correcting the directory owner to 10001 repairs this specific permission mismatch without making it writable by every user. This lab uses rootful Docker without user-namespace remapping; mappings and host security policy can add other permission checks in other environments.

A read-only volume mount independently prevents writes even when ownership is correct. `chmod` or `chown` cannot override that mount property. Inspect the mount's RW field and compare the exact error; Permission denied and Read-only file system point to different layers.

Mounts can hide files that exist at the same path in the image. A missing file therefore does not always mean deletion. This fixture uses a dedicated `/data` path and deliberately compares a volume file with `/ephemeral.txt` outside that mount.

Source: docs/chaos-theory.md: §§5.13.3-5.13.4.

The experiment: Use only the named volume `ce-lab18-data` and disposable `ce-lab18` containers. Setup resets this exercise's data; save observations elsewhere.

How to read the evidence

Signal	Meaning
UID / GID / directory mode	id and numeric ls output show the identity and permissions used by the kernel, even if no account name exists.
Mounts / RW	Docker inspect identifies the named volume and whether this container mounted it writable. Volume existence alone does not prove write access.
ephemeral.txt / persisted.txt	Compare paths after replacing the container, not merely restarting the original one.
write exit / error	Read the failing command result directly. A successful later ls must not hide the earlier write failure.

Predict

Predict which file survives replacement and whether UID 10001 can write before chown, after chown and with a read-only mount.

Record your results

Phase	Files visible	Owner / mount RW	Write result
Original container			
Replacement container			
UID mismatch			
Owner corrected			
Read-only mount			
Writable recovery			

Procedure

1. Put one file in each storage layer

VM terminal 1

```
docker run --name ce-lab18-original --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c
'echo layer > /ephemeral.txt; echo volume > /data/persisted.txt'
docker cp ce-lab18-original:/ephemeral.txt ~/labs/lab18/original-layer.txt
docker inspect -f '{{json .Mounts}}' ce-lab18-original
docker rm ce-lab18-original
docker run --rm --name ce-lab18-check --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim sh -c
'cat /data/persisted.txt; test ! -e /ephemeral.txt && echo writable-layer-file-absent'
```

2. Diagnose a numeric ownership mismatch

VM terminal 1, same shell

```
docker run --rm --name ce-lab18-check --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data
python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
```

3. Repair only ownership and repeat the same write

VM terminal 1, same shell

```
docker run --rm --name ce-lab18-check --mount type=volume,src=ce-lab18-data,dst=/data python:3.12-slim chown
10001:10001 /data
docker run --rm --name ce-lab18-check --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data
python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
```

4. Change only mount writability, inspect, then recover

VM terminal 1, same shell

```
docker run --name ce-lab18-readonly --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data,readonly
python:3.12-slim sh -c 'id; ls -ldn /data; echo attempt > /data/user.txt'
echo "write exit=$?"
docker inspect -f '{{json .Mounts}}' ce-lab18-readonly
docker rm ce-lab18-readonly
docker run --rm --name ce-lab18-check --user 10001:10001 --mount type=volume,src=ce-lab18-data,dst=/data
python:3.12-slim sh -c 'echo recovered > /data/user.txt; cat /data/user.txt /data/persisted.txt'
docker volume rm ce-lab18-data
```

Recovery check: A write succeeds again after restoring a writable mount, and the dedicated test volume is removed only after evidence has been recorded.

Answer

Explain each result using the storage lifetime, numeric ownership and mount mode. Show a successful write after the narrow repair and show that read-only still blocks it.

Check your understanding

Would `chmod 777` repair a read-only mount, and would it be an appropriate first response to an ownership mismatch?

Save results before removing this lab's resources and files: `./lab.sh 18 reset`

Lab 19 - Unscheduled Pods versus memory-killed containers

Did the workload fail before placement or after exceeding its memory limit?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 19 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

Requests and limits answer different questions. The scheduler accounts for requested resources against node allocatable capacity and existing requests. It does not place a Pod merely because live CPU usage looks low. CPU 20m means 0.02 CPU; CPU 1000 means one thousand CPUs, not one thousand millicores.

An impossible request leaves this Pod unassigned with `PodScheduled=False` and reason `Unschedulable`. No application container has started, so application logs cannot diagnose its execution. Scheduling events describe constraints; inspect the current Pod UID because a previous object may have used the same name.

A memory limit is enforced at runtime through the container's memory cgroup. Kubernetes Mi means MiB (1048576 bytes). The OOM fixture requests 16 MiB but has a 32 MiB limit and deliberately touches 96 MiB. Its request fits; its allocation does not. A memory request is not the maximum amount the application may use.

`OOMKilled` in `lastState.terminated`, an exit code and a rising `restartCount` identify a different stage from `Unschedulable`. Exit 137 alone cannot identify OOM. `CrashLoopBackOff` describes a delay before repeated restarts, not their root cause. Inspect the previous instance's logs while the Pod still exists.

These standalone Pods use `restartPolicy Always` by default. Kubelet can restart a container inside the same Pod UID. The procedure intentionally deletes and recreates the Pod between configurations because scheduling and resource fields cannot all be edited in place. That intentional UID change is not evidence of self-healing by a Deployment.

Repair the demonstrated cause. Restarting cannot make a 1000-CPU request fit. Repeated restarts cannot make the same 96 MiB allocation fit a 32 MiB limit. In a real workload, investigate memory growth before choosing a measured limit; simply removing limits can move the failure to the node.

Source: docs/chaos-theory.md: §§10.6.1-10.6.2.

The experiment: Use namespace `ce-lab19` on this lab's chaos cluster. Compare a healthy HTTP process, an impossible CPU request and a bounded memory allocation; then restore the healthy fixture.

How to read the evidence

Signal	Meaning
nodeName / PodScheduled / events	Distinguish no placement from a node-assigned runtime problem. Events are filtered by this Pod UID.
requests / limits / allocatable	Read configured quantities and node capacity. No metrics-server is required for this diagnosis.
lastState.terminated / restartCount	The previous container result and restart count remain attached to this Pod. Capture them before deleting it.
logs --previous	Reads the previous container instance, not a previous Pod with the same name. Missing logs are not proof that no failure happened.
k19	kubect! using Lab 19's private kubeconfig, kind-lab19 context and only namespace ce-lab19.

Predict

Predict node assignment, available logs and restart evidence for the impossible request and the memory limit failure.

Record your results

Phase	UID / node	Scheduled / Ready	Reason / restarts
Healthy			
1000 CPU request			
32 MiB limit / 96 MiB allocation			
Healthy restored			

Procedure

1. Inspect a healthy placement and its configured resources

VM terminal 1

```
source ~/labs/lab19/helpers.sh
k19 get nodes -o
  custom-columns='NAME:.metadata.name,CPU:.status.allocatable.cpu,MEMORY:.status.allocatable.memory'
k19 get pod resource-web -o json | jq
  '{uid:.metadata.uid,node:.spec.nodeName,resources:.spec.containers[0].resources,status:.status}'
k19 logs resource-web
```

2. Replace it with an impossible request and diagnose placement

VM terminal 1, same shell

```
k19 delete pod resource-web --wait=true --timeout=30s
k19 apply -f ~/labs/lab19/unscheduled.yaml
k19 wait pod/resource-web --for=jsonpath='{.status.conditions[?(@.type=="PodScheduled")].reason}'=Unschedulable
  --timeout=60s
k19 get pod resource-web -o json | jq
  '{uid:.metadata.uid,node:.spec.nodeName,resources:.spec.containers[0].resources,conditions:.status.condition
s}'
uid=$(k19 get pod resource-web -o jsonpath='{.metadata.uid}')
k19 get events --field-selector "involvedObject.uid=$uid"
k19 logs resource-web --request-timeout=5s
```

3. Replace it with a schedulable memory failure and preserve crash evidence

VM terminal 1, same shell

```
k19 delete pod resource-web --wait=true --timeout=30s
k19 apply -f ~/labs/lab19/oom.yaml
k19 wait pod/resource-web --for=jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'=OOMKilled
--timeout=90s
k19 get pod resource-web -o json | tee ~/labs/lab19/oom-evidence.json | jq
'{"uid":.metadata.uid,node:.spec.nodeName,resources:.spec.containers[0].resources,containers:.status.container
Statuses}'
k19 logs resource-web --previous | tee ~/labs/lab19/previous.log
uid=$(k19 get pod resource-web -o jsonpath='{.metadata.uid}')
k19 get events --field-selector "involvedObject.uid=$uid"
```

4. Restore a healthy workload and verify recovery

VM terminal 1, same shell

```
k19 delete pod resource-web --wait=true --timeout=30s
k19 apply -f ~/labs/lab19/good.yaml
k19 wait pod/resource-web --for=condition=Ready --timeout=60s
k19 get pod resource-web -o json | jq
'{"uid":.metadata.uid,node:.spec.nodeName,containers:.status.containerStatuses}'
```

Recovery check: The healthy resource-web Pod is Ready, and its current container has no OOM termination or repeated restarts.

Answer

Diagnose the earliest failed stage in each case using conditions, current-UID events, resources, termination reason and logs. Show that the restored Pod is Ready.

Check your understanding

Would increasing the CPU limit fix an unschedulable Pod whose CPU request exceeds every node's capacity?

Save results before removing this lab's resources and files: `./lab.sh 19 reset`

Lab 20 - Trace DNS, Service ports and the application listener

At which layer does a failed in-cluster HTTP request break?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 20 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

Follow the request through name resolution, Service IP and port, endpoint address and target port, then the application's listener. The Service selector matches Pod labels; EndpointSlices list those backend addresses, ports and readiness conditions. A DNS answer identifies an address; it does not establish that HTTP will succeed.

An ordinary ClusterIP Service has a DNS name scoped by namespace. `web.ce-lab20.svc.cluster.local` identifies this fixture in kind's cluster.local domain. A wrong namespace in the name can fail resolution while the correct Service IP still works. Diagnose that difference before changing the cluster DNS server.

Service port is the client-facing port; targetPort is the backend port. This Service accepts port 80 and forwards to 8080. Changing targetPort to 8099 keeps DNS, selectors and Pod readiness intact, but forwards to a port with no server. containerPort metadata does not open a socket or rewrite an application's listen port.

Compare HTTP by Service name, Service IP and direct Pod IP from the same client Pod. Name failure with IP success points to resolution or naming. Service failure with direct Pod success points toward Service selection, port mapping or routing; the endpoint and port evidence narrows the cause.

A listener bound to 127.0.0.1 accepts only connections inside that Pod's network namespace. It can answer a localhost request from kubectl exec while refusing the client Pod's request to its Pod IP. The HTTP readiness probe also uses the Pod IP by default, so this bind fault makes the server unready and removes ordinary Service eligibility.

A timeout is not a diagnosis of a NetworkPolicy or DNS failure. Inspect the layer actually tested. This fixture does not install a NetworkPolicy-enforcing CNI and makes no NetworkPolicy claims. Other real causes include application failure, firewall rules and CNI routing.

Source: docs/chaos-theory.md: §§10.7.1-10.7.2.

The experiment: Use namespace ce-lab20 with one Python server and one Python client. Change the requested name, then targetPort, then the bind address; restore each fault before the next comparison.

How to read the evidence

Signal	Meaning
resolve / probe	Helpers run socket DNS lookup or a bounded HTTP request inside the same client Pod. Errors return nonzero and are printed, not counted as healthy HTTP.
Service ports / EndpointSlices	Compare the requested Service port with the endpoint port and readiness. A nonempty endpoint list can still contain the wrong port.
localhost versus Pod IP	Local success proves a local listener; it does not prove that another Pod can reach that listener.
k20	kubectl fixed to kind-lab20 and ce-lab20. The client remains unchanged while the server is intentionally recreated for the bind comparison.

Predict

Predict DNS, Service-IP HTTP, direct-Pod HTTP and readiness for the wrong name, wrong targetPort and loopback-only listener.

Record your results

Phase	DNS	Service / Pod HTTP	Ready / endpoint port
Baseline			
Wrong namespace name			
Wrong targetPort			
Port restored			
Loopback listener			
Listener restored			

Procedure

1. Establish the same-client baseline at every layer

VM terminal 1

```
source ~/labs/lab20/helpers.sh
SVC_IP=$(k20 get svc web -o jsonpath='{.spec.clusterIP}')
POD_IP=$(k20 get pod web -o jsonpath='{.status.podIP}')
resolve web.ce-lab20.svc.cluster.local
probe http://web.ce-lab20.svc.cluster.local/
probe "http://$SVC_IP/"
probe "http://$POD_IP:8080/"
k20 get svc web -o json | jq '.spec.ports'
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
```

2. Test a wrong namespace name without changing the cluster

VM terminal 1, same shell

```
resolve web.ce-lab20-missing.svc.cluster.local
probe http://web.ce-lab20-missing.svc.cluster.local/
probe "http://$SVC_IP/"
probe http://web.ce-lab20.svc.cluster.local/
```

3. Change only targetPort and compare Service and direct traffic

VM terminal 1, same shell

```
k20 patch svc web --type=json -p ' [{"op": "replace", "path": "/spec/ports/0/targetPort", "value": 8099}] '
sleep 3
resolve web.ce-lab20.svc.cluster.local
k20 get pod web
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
probe "http://$SVC_IP/"
probe "http://$POD_IP:8080/"
k20 patch svc web --type=json -p ' [{"op": "replace", "path": "/spec/ports/0/targetPort", "value": 8080}] '
sleep 3
probe http://web.ce-lab20.svc.cluster.local/
```

4. Bind the server only to localhost and locate the failed boundary

VM terminal 1, same shell

```
k20 delete pod web --wait=true --timeout=30s
k20 apply -f ~/labs/lab20/loopback.yaml
k20 wait pod/web --for=jsonpath='{.status.phase}'=Running --timeout=60s
sleep 3
POD_IP=$(k20 get pod web -o jsonpath='{.status.podIP}')
k20 exec web -- python -c 'import urllib.request;
    print(urllib.request.urlopen("http://127.0.0.1:8080/").status) '
probe "http://$POD_IP:8080/"
k20 get pod web -o json | jq '{command:.spec.containers[0].command,conditions:.status.conditions}'
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
probe http://web.ce-lab20.svc.cluster.local/
```

5. Restore the listener and prove the full path works

VM terminal 1, same shell

```
k20 delete pod web --wait=true --timeout=30s
k20 apply -f ~/labs/lab20/server.yaml
k20 wait pod/web --for=condition=Ready --timeout=60s
for i in $(seq 1 10); do probe http://web.ce-lab20.svc.cluster.local/ && break; sleep 1; done
k20 get endpointslices -l kubernetes.io/service-name=web -o json | jq '.items[] | {ports,endpoints}'
probe http://web.ce-lab20.svc.cluster.local/
```

Recovery check: DNS and Service HTTP work, targetPort is 8080, and the server listens on 0.0.0.0 with a Ready endpoint.

Answer

Build a layer-by-layer diagnosis from the same client. Explain why DNS success and local HTTP success are weaker than successful Service HTTP, and show recovery after every fault.

Check your understanding

If DNS works but direct Pod-IP HTTP also fails, is changing only the Service selector a justified repair?

Save results before removing this lab's resources and files: `./lab.sh 20 reset`

Lab 21 - Configuration freshness and a stalled rollout

Why can a ConfigMap update leave old behavior running, and why can a rollout fail while the application stays available?

Before you run: Provision the shared VM once with `./lab.sh provision`. Run `./lab.sh 21 setup`, then `./lab.sh ssh`. No other lab is required.

Theory you need

A ConfigMap is a separate API object. This application reads its message through an environment variable when its container starts. Updating the ConfigMap does not rewrite the environment of existing processes and does not change the Deployment's Pod template, so it does not itself create a rollout.

A projected ConfigMap volume behaves differently. Ordinary projected files can update eventually, but the application must reread or reload them; a subPath mount does not receive those updates. Do not infer environment refresh from file-projection behavior. This lab tests environment injection specifically.

A Deployment rollout occurs when its Pod template changes. rollout restart changes a template annotation and creates new Pods; they read the current ConfigMap. Record Pod UIDs and the actual HTTP response from each Pod so a mixture of old and new processes cannot hide behind a single sampled response.

This Deployment has two replicas, `maxUnavailable=0` and `maxSurge=1`. During this update it can add one Pod while keeping two replicas available. `minReadySeconds=2` requires a new Pod to remain Ready for two seconds before it counts as available. The strategy governs replacement; a `PodDisruptionBudget` does not control this rollout.

A wrong readiness port can leave a new process Running but unready. It must not replace an available old replica under this strategy. When no rollout progress occurs for the configured 30-second deadline, the Deployment reports `ProgressDeadlineExceeded`. Kubernetes reports the stall; it does not automatically undo the change.

Rollout history stores Pod-template revisions, not snapshots of separate ConfigMaps. Undo to the recorded good revision repairs the bad readiness port, but does not restore the ConfigMap's old message. Recovery requires checking both template and configuration, then confirming the intended responses and replica counts.

Source: docs/chaos-theory.md: §§10.8.1-10.8.2.

The experiment: Use namespace `ce-lab21`, two Python HTTP replicas, an environment-backed message and a deliberately wrong readiness port. Query every Pod directly through the API proxy; this measures application responses, not Service routing.

How to read the evidence

Signal	Meaning
ConfigMap value / per-Pod HTTP	Compare desired configuration with each running process. An updated object is not proof that every process consumed it.
UID / revision / ReplicaSets	UID changes show replacement. The Deployment revision identifies a Pod template; ReplicaSet desired/current/ready counts show rollout progress.
Available / Progressing	Availability of old replicas and progress of the new template are separate conditions. Both must be interpreted with the replica counts.
readiness port / restartCount	A bad readiness destination can block rollout without crashing or restarting the process.
k21 / responses	kubect! targets kind-lab21 and ce-lab21. responses prints each Pod name and its HTTP body; a missing or failed query returns nonzero.

Predict

Predict Pod UIDs and response values after only the ConfigMap changes, after restart, during the bad probe rollout and after undo.

Record your results

Phase	ConfigMap / responses	UIDs / Ready replicas	Revision / conditions
Version one			
ConfigMap version two only			
Restarted consumers			
Wrong readiness port			
Undo to stable revision			
Version one restored			

Procedure

1. Record configuration, identities and actual responses

VM terminal 1

```
source ~/labs/lab21/helpers.sh
k21 get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k21 get pods -l app=config-web -o
  custom-columns='NAME:.metadata.name,UID:.metadata.uid,READY:.status.conditions[?(@.type=="Ready")].status'
responses
```

2. Change only the ConfigMap and query the same processes

VM terminal 1, same shell

```
k21 patch configmap app-config --type=merge -p '{"data":{"message":"version-two"}}'
k21 get configmap app-config -o jsonpath='{.data.message}{"\n"}'
k21 get pods -l app=config-web -o custom-columns='NAME:.metadata.name,UID:.metadata.uid'
responses
```

3. Roll out new consumers and save the good template revision

VM terminal 1, same shell

```
k21 rollout restart deployment/config-web
k21 rollout status deployment/config-web --request-timeout=0 --timeout=120s
responses
stable_revision=$(k21 get deployment config-web -o json | jq -r
'.metadata.annotations["deployment.kubernetes.io/revision"]')
echo "stable revision=$stable_revision"
k21 rollout history deployment/config-web
```

4. Break only the new template's readiness port and diagnose the stall

VM terminal 1, same shell

```
k21 patch deployment config-web --type=json -p
'[{ "op": "replace", "path": "/spec/template/spec/containers/0/readinessProbe/httpGet/port", "value": 9999 } ]'
k21 wait deployment/config-web
--for=jsonpath='{.status.conditions[?(@.type=="Progressing")].reason}'=ProgressDeadlineExceeded
--timeout=90s --request-timeout=0
k21 rollout status deployment/config-web --timeout=5s
k21 get deployment config-web -o json | jq '{desired:.spec.replicas,status:.status}'
k21 get rs -l app=config-web
k21 get pods -l app=config-web -o json | jq '.items[] |
{name:.metadata.name,uid:.metadata.uid,probe:.spec.containers[0].readinessProbe,conditions:.status.conditions,
restarts:.status.containerStatuses[0].restartCount}'
responses
```

5. Undo the bad template and inspect the separate configuration

VM terminal 1, same shell

```
k21 rollout undo deployment/config-web --to-revision="$stable_revision"
k21 rollout status deployment/config-web --request-timeout=0 --timeout=120s
k21 get configmap app-config -o jsonpath='{.data.message}'{"\n"}'
responses
k21 get deployment config-web -o
jsonpath='{.spec.template.spec.containers[0].readinessProbe.httpGet.port}'{"\n"}'
```

6. Restore configuration explicitly and verify every consumer

VM terminal 1, same shell

```
k21 patch configmap app-config --type=merge -p '{"data":{"message":"version-one"}}'
k21 rollout restart deployment/config-web
k21 rollout status deployment/config-web --request-timeout=0 --timeout=120s
sleep 3
k21 get pods -l app=config-web
responses
k21 get deployment config-web -o json | jq
'{desired:.spec.replicas,ready:.status.readyReplicas,conditions:.status.conditions}'
```

Recovery check: The ConfigMap says version-one, exactly two Pods are Ready, both HTTP responses say version-one, and the readiness port is 8080.

Answer

Explain configuration freshness and rollout progress separately. Record the stable revision before the fault, diagnose the new Pod, show old replicas remain available, and prove what undo did and did not restore.

Check your understanding

Would undo to the original Pod-template revision necessarily restore the original environment value from the ConfigMap?

Save results before removing this lab's resources and files: `./lab.sh 21 reset`

Solutions

Compare after recording your own results. The expected patterns below assume a working baseline and a confirmed injection. Differences require analysis; do not replace observations with the expected answer.

Lab 00 - Prepare the learning environment

A usable baseline lets you attribute later changes to the fault instead of a broken setup. Version records make results interpretable; the saved baseline supports comparisons.

Step 1: cgroup2fs, Docker cgroups=2, a successful hello-world run, and at least 60 GiB total root capacity.

Step 2: The commit is 3e3ee64db71f51a5e9f79af8562dd4aa913a0e71. Record the local image ID and the registry digest for each pulled image; the published Goldpinger image tag is 3.11.3, without a v prefix.

Understanding check: hello-world checks shared container tooling. It does not test the Redis service, its listener or the client path used by Lab 01.

Lab 01 - A timeout makes a silent failure manageable

REJECT should reach the handler quickly. DROP without an application timeout should reach the five-second watchdog instead. With TIMEOUT=0.5, the client should handle the timeout near half a second. The handler needs an error; it does not create a time limit.

Step 1: PONG, then OK True.

Step 2: REJECT prints DEGRADED ConnectionError within milliseconds. DROP without a timeout prints nothing and is stopped by the external watchdog (exit=124). DROP with TIMEOUT=0.5 prints DEGRADED TimeoutError near 0.5 s. The ce-lab01 rule shows pkts above 0. The final client prints OK True.

Understanding check: No. Attempt budgets and any backoff accumulate. A whole-request deadline must also cover retries and other work.

Lab 02 - Prove why a process was killed

The allocator should trigger a cgroup OOM kill before its runtime bound. Status 137 is consistent with SIGKILL, but only the matching memory-kill evidence explains why. If the logs show a runtime timeout or a userspace kill, report that cause instead.

Step 1: Bash normally reports 137. Here you know the cause because the program deliberately sent SIGKILL.

Step 2: The unit fails before the 20 s bound, with oom-kill in the unit log and a matching kernel line. An oomd entry means userspace acted instead.

Understanding check: No. The allocator is constrained by its cgroup boundary. VM-wide availability and the cgroup's remaining allowance are different quantities.

Lab 03 - Restart policies have limits

Restart=always requests a restart, but the start-rate limit can refuse it. One crash can remain within the allowance; repeated starts can exhaust it and leave A failed. The journal, not a successful response through B, establishes what happened to A. If the limit was not reached, report that observation rather than claiming it was.

Step 2: A should return to active. A successful proxy response alone cannot prove that A restarted.

Step 3: Look for "Start request repeated too quickly" and a failed unit. Result and restart counts can vary with timing and systemd version; check the journal.

Understanding check: It would test whether the rate limit caused the supervisor to give up. The application would still crash, and rapid restarts would still consume resources and need monitoring.

Lab 04 - CPU contention and a quota

An unrestricted neighbour competes for CPU time. Capping it should leave more time for work.py, though the measured gain depends on scheduling. `cpu.max` shows the configured cap; `cpu.stat` shows throttling. Recovery toward baseline strengthens the contention explanation. A run that outlasts the neighbour mixes contention with recovery and cannot support the comparison.

Step 1: One CPU permitted by this shell's affinity; use the same shell and CPU for every phase.

Step 3: Compare iteration time with baseline. PSI and `vmstat` provide supporting waiting measurements; neither alone proves which task caused the slowdown.

Step 4: `cpu.max` should show a quota/period ratio of 0.2, and `cpu.stat` should show throttling. If the neighbour ends before measurement finishes, shorten work.py's iteration count and repeat every phase with that same workload.

Understanding check: No. It changes placement and removes the intended competition for a single CPU. Keep placement fixed when evaluating the quota.

Lab 05 - A container is several Linux mechanisms

The prepared root explains the files under /. The PID namespace explains the different process view. The cgroup explains the CPU cap and throttling. None of these creates a separate kernel or automatically isolates every shared resource.

Step 3: `$$` is 1, `ps` lists only the sandbox processes, and / is the BusyBox tree.

Step 4: `cpu.max` 20000 100000, `memory.max` 134217728, `pids.max` 50; `lsns` shows a separate PID namespace whose command is `sh`.

Step 6: `nr_throttled` and `throttled_usec` increase; `top` shows `sh` near 20 percent CPU.

Understanding check: No. The PID view is controlled by the PID namespace and `proc` mount. Removing a bandwidth limit changes resource consumption, not visibility.

Lab 06 - Separate containers can share a full filesystem

The new container has a separate process environment but uses the same full tmpfs. Its write therefore fails until the filling file is removed. Success before filling and after freeing space supports shared-capacity exhaustion as the explanation. Container separation alone does not protect this shared resource.

Step 1: The one-MiB write succeeds before space is consumed.

Step 2: The filling write and the second container's probe reach No space left on device.

Step 3: The same probe succeeds after capacity is freed.

Understanding check: Its file is stored in the host bind mount, outside the disposable container layer. Remove that file or the temporary filesystem to release the capacity.

Lab 07 - Network delay accumulates across sequential calls

After baseline subtraction, one exchange should gain roughly 25 ms and four roughly 100 ms. Each reply is a separate dependent wait. Counters confirm the altered queue carried traffic, while recovery checks the attribution. The `tc` process installs a persistent namespace setting: its exit does not undo it. Timing noise, packet behaviour and scheduling can cause deviations; report your measurements.

Step 1: The client receives every echo; `eth0` has its default `noqueue`, with no custom root `qdisc`. Stop if either check fails.

Step 2: `netem` remains installed after `tc` exits, with traffic counters. Added time should be small; record the range.

Step 3: Four sequential exchanges should accumulate more delay than one. Compare baseline-subtracted times, not raw ratios.

Step 4: No `netem` remains and timing returns toward baseline.

Understanding check: No. Sequential dependency waits add along a request's critical path. Independent parallel waits can overlap; other overhead still needs measurement.

Lab 08 - A syscall error tests application handling

This server treats a close error as fatal and should exit with status 1. The triggering request may already have received its response, but a dead process cannot serve later requests. The fault exposes an application error-handling decision, not a Kubernetes or network failure.

Step 1: HTML lines from the server. Keep this terminal: wait "\$server" only works in it.

Step 4: Many write calls, one fsync and one close per request. fsync on a socket returns -1 EINVAL, which is not an outage cause.

Step 6: strace shows close(...) = -1 EIO (Input/output error) (INJECTED); the server exits with status 1 after "error closing socket". The next request should fail to connect; record its result separately from the first response.

Step 7: HTTP responds after restart; the test server is then stopped.

Understanding check: No. strace can skip the real call and substitute an error. Real close errors have syscall-specific semantics. The supported conclusion is about this application's reaction to the injected return value.

Lab 09 - A syscall denial need not crash the process

Before the filter, getpid returns the process ID. Afterward, the filter makes it return -1 with errno 13 (EACCES). Printing still works because the filter allows other syscalls. The program's exit 0 reports a successful check of the denial, not a successful getpid call. Exit 1-3 means filter setup failed; exit 4 means the expected denial was not observed.

Step 1: Before filtering, getpid prints a positive PID. After filtering, getpid result=-1 errno=13 and exit=0. Exit 1-3 means filter setup failed; 4 means the expected denial did not occur.

Step 2: This fresh process has its own Seccomp state. An existing inherited filter can be present; the test did not install a host-wide policy.

Understanding check: No. It frees the library context in userspace. The kernel filter remains installed; the demonstration's later getpid call tests exactly this.

Lab 10 - Ownership, disruption budgets and Service routing

The ReplicaSet creates a replacement because the desired replica count remains three. Remaining backends may serve requests during that work, but endpoint changes and in-flight requests can still produce failures. A replacement proves reconciliation; the probe results measure sampled service availability. Zero failed samples means no failure was observed by these probes, not that every possible request succeeded. The ownership chain explains replacement. The PDB rejects an eviction but does not intercept direct DELETE. A selector mismatch isolates a discovery error: Ready processes remain, but no matching backends can serve the Service.

Step 1: Three Ready replicas, matching endpoints and a working /healthz probe. The workload may list Pods but may not delete them.

Step 2: The eviction is refused because it would violate the budget; the original UID still exists. Stop if the error is Forbidden or if eviction succeeds, since those do not establish the intended comparison.

Step 5: The victim is NotFound, a new pod name replaced it, all READY=True, three endpoints and a counted number of non-200 samples (0 or a few, right after the deletion).

Step 6: Ready Pods can coexist with zero eligible Service endpoints and a failed request. Recheck endpoints if updates are still converging.

Step 7: Three eligible endpoints and successful HTTP return without a workload rollout.

Understanding check: No. Replica count does not guarantee placement across failure domains. This experiment deletes one Pod; it does not test node or zone loss.

Lab 11 - Health depends on what you test

The 100 ms dose should increase observed peer latency while leaving enough of the 300 ms budget to succeed. At 400 ms the same operation should time out. The TCP listener can still accept connections, so readiness can remain true in both cases. Removing the toxic should restore peer success without a Pod restart. Boolean health has a threshold; readiness only describes its configured test. HTTP readiness still disagrees with peers at 400 ms because its deadline is longer. At 1400 ms it also fails, removing eligibility without a restart. Recovery within the same Pod demonstrates a readiness transition, not a replacement. Direct Pod discovery is not filtered by Service endpoint readiness.

Step 1: The extra replica is Ready and peer reports for its IP show OK true. Establish this baseline before adding delay.

Step 2: With enough margin, peer pings remain healthy but take longer. Readiness should remain true; record actual responses.

Step 3: Look for failed peer pings while the Pod remains Ready. The 400 ms delay exceeds the 300 ms ping budget; readiness only checks TCP connection establishment.

Step 4: Record a fresh UID for the HTTP configuration and a healthy baseline before injecting again.

Step 5: Peer requests can fail at 300 ms while HTTP readiness still succeeds within one second.

Step 6: The slow Pod becomes unready and its endpoint is not ready. UID and restart counts stay unchanged; other Service endpoints remain available.

Step 7: No toxic remains, peer pings recover, and the original three replicas are Ready after the extra Pod is deleted.

Understanding check: No. A healthy process can fail a dependency-based liveness check and restart repeatedly without repairing that dependency. Readiness controls routing; liveness should detect a condition a restart can plausibly fix.

Lab 12 - Locate startup failures and measure recovery

The delayed workload should miss the 30-second budget and be reported as unsuccessful. Restored healthy runs check that the fault was removed. These trials validate those checker paths; they do not establish an SLO for all future startups. The unscheduled trial fails before a container can start; the selector trial can have a Ready process but no usable Service backend. Diagnose using conditions and endpoints captured before cleanup. Report healthy successes divided by three completed trials without mixing in deliberate failures.

Step 1: One result with success=1 and a few seconds elapsed; both final gets report NotFound. A nonzero checker exit or no new result means the check did not complete; do not interpret old metrics as a new result.

Step 2: The grep shows the sleep 45 command; the run reports success=0 after about 30 s.

Step 3: A completed miss should show PodScheduled=False/Unschedulable with no assigned node. This is not evidence of a slow web process.

Step 4: A Ready Pod with no matching endpoints isolates Service selection as the failure stage.

Understanding check: No. It checks the registry for the image resolution, but cached layers can still be reused. A cold-image experiment must control the node's cache as well.

Lab 13 - Placement, supervision and replacement

Missing kubelet reports trigger detection and eviction, not an immediate process kill. Replacement includes several waits, while the old runtime may keep serving its container. Restoring kubelet allows that node to reconcile and clean up. Cordon only changes eligibility for new scheduling; deleting the managed Pod causes replacement elsewhere. Uncordon does not rebalance it. Missing Lease renewals after stopping kubelet precede node-unavailable handling, so toleration duration is only one part of the replacement delay.

Step 1: One Ready node-web Pod on a worker; the two NoExecute tolerations explicitly say 20 seconds. Save its container ID.

Step 2: The same UID and container remain while the node is cordoned; kubelet still runs.

Step 3: Replacement runs on another eligible node; uncordon does not migrate it. Record the new original identity for the separate kubelet experiment.

Step 4: The original container can remain running as the node becomes unavailable and a replacement starts elsewhere. Detection and replacement are separate events. If none appears within the bound, report that and recover.

Step 5: Four Ready nodes. If a replacement was created, the old container should disappear as kubelet reconciles. Recheck after a short wait if it is still terminating.

Understanding check: Removing an API object does not fence or stop an unreachable machine. The old process can continue; preventing concurrent writers requires an appropriate fencing or application coordination mechanism.

Lab 14 - Quorum, convergence and application availability

Two surviving members can commit; one cannot. Existing HTTP traffic may continue because it does not need a new etcd write for each request. Brief transition errors are possible even with quorum. Restoring these members tests temporary quorum recovery, not recovery from permanent data loss. With quorum, desired replicas can change and controllers can converge to three. Without it, the experiment cannot reliably commit another scale target, although existing traffic may work. Read persisted state after recovery before setting the explicit two-replica baseline. An unchanged leader after losing a follower is expected, not failed fault injection.

Step 1: Four Ready nodes, 2/2 web pods, HTTP ok, "configmap/quorum-probe patched" and a table with three members, one IS LEADER=true.

Step 2: No etcd container on lab14-control-plane2; the write normally succeeds because two of three members keep quorum (a single failure can come from the stopped member's API server before the load balancer drops it, so retry once); HTTP returns 200. Scaling to three should commit and converge while quorum survives. Endpoint status from a surviving member identifies the leader.

Step 3: With only one voter, no new writes can commit. Record client errors and any readable state separately. Existing HTTP may continue; a timeout is not a general proof that a submitted operation never committed.

Step 4: Record persisted values before the next step. Failed requests during quorum loss normally leave one-down and three replicas, but the observed state resolves any client uncertainty.

Step 5: All three endpoints are healthy; the recovered value is readable, desired and Ready replicas are both two, and HTTP works.

Understanding check: No. Four members require three votes and tolerate one failure, just as three members tolerate one. Five members require three and tolerate two.

Lab 15 - Count how layered retries multiply work

Healthy phases should make 20 client attempts and 20 backend arrivals, with 20 successes. During persistent 503, no client retry allows 20 client attempts and up to 40 backend arrivals; one client retry allows 40 and up to 80. Both can still yield zero user successes. These are predicted counts for this controlled failure, not universal amplification factors. Recovery demonstrates that retries stop on success. This experiment establishes multiplication of work, not a self-sustaining overload.

Step 1: Three phase summaries, each with 20 original requests. Check the baseline before using the fault result.

Step 2: The fault can produce twice as many backend arrivals, still without a successful original request. Healthy and recovered phases should stop after their first successful attempt.

Step 3: No listeners remain on the three lab ports. Logs and summaries remain available for your answer.

Understanding check: Three retries means four attempts at each layer, so at most $4 \times 4 = 16$ backend arrivals per original request, provided every attempt reaches the backend and keeps failing.

Lab 16 - Design one clear experiment

A complete experiment connects one controlled fault to one measurable outcome. It also states how to stop and restore the service. If the outcome cannot be measured or the fault cannot be removed, improve the design before considering a run.

Step 2: No blank fields remain. Read the answers too: this check does not validate their meaning.

Understanding check: No. The card is a plan, and the control establishes the measurement path and baseline. Only an observed fault trial and analysis can support the specific resilience claim.

Lab 17 - Signal delivery and the shutdown budget

The wrapper receives Docker's TERM but intentionally does not forward it, so the child never starts its handler. Exec delivers TERM to the worker and five seconds allows its two-second cleanup to complete. With one second the handler starts but cannot finish before forced termination. These are delivery and timing failures, not evidence of OOM.

Step 1: The worker reports a PID other than 1. Do not stop it before the ready line appears.

Step 2: No handler receipt or completion marker, a forced exit and OOMKilled=false. The missing-file copy error is expected evidence.

Step 3: PID 1 receives TERM, completes cleanup and exits zero.

Step 4: TERM receipt appears but cleanup completion does not. Record evidence before removing the containers.

Understanding check: No. More time cannot make this wrapper forward a signal it ignores. Repair the process relationship or implement correct forwarding first.

Lab 18 - Persistence, ownership and read-only mounts

The named-volume file survives container replacement while the writable-layer file does not. UID 10001 initially lacks directory write permission. Assigning the intended owner repairs that case; the same user then fails when the volume is mounted read-only. Repair the layer that the evidence identifies rather than making all files world-writable.

Step 1: The volume file survives replacement. The file outside the volume exists in the original container but not its replacement.

Step 2: The directory is owned by 0 and the non-owner write fails with Permission denied.

Step 3: The owner is now 10001 and the write succeeds without chmod 777.

Step 4: Correct ownership still cannot write through RW=false. Restoring a writable mount restores the write and existing data remains readable.

Understanding check: No. Mount read-only enforcement is independent of Unix mode bits. For an ownership mismatch, inspect the required identity and grant only the needed access.

Lab 19 - Unscheduled Pods versus memory-killed containers

The impossible request fails during scheduling and has no running application to debug. The OOM fixture is scheduled, starts and exceeds its runtime memory bound; kubelet restarts it within the same UID. Use the recorded reason and previous logs to explain the crash rather than treating CrashLoopBackOff as a diagnosis.

Step 2: No assigned node; scheduling evidence reports insufficient CPU on eligible workers. Logs cannot show an application that has not started.

Step 3: A node is assigned; the previous container is OOMKilled and the allocation message identifies the deliberately oversized work. Recheck if the restart status is still updating.

Step 4: A new deliberately recreated Pod is Ready. The fault evidence remains in the saved files.

Understanding check: No. Placement uses the request and available allocatable capacity. Increasing a limit does not reduce that request or add a suitable node.

Lab 20 - Trace DNS, Service ports and the application listener

A wrong namespace breaks naming while the correct IP path remains usable. The targetPort fault leaves the backend healthy but misdirects Service traffic. The loopback bind leaves local HTTP working while remote Pod-IP access and readiness fail. The separate probes locate different stages instead of treating every connection failure as a DNS problem.

Step 2: The incorrect name fails while the correct name and Service IP still work. No DNS server restart is needed.

Step 3: DNS and direct Pod HTTP work, but the wrong endpoint port breaks the Service request. Restoring 8080 repairs it.

Step 4: Local HTTP can succeed while remote Pod HTTP fails and readiness remains false. This is not a successful service baseline.

Understanding check: Not from that evidence. The direct request bypasses the Service selector. Investigate the process listener, destination port and Pod network path first.

Lab 21 - Configuration freshness and a stalled rollout

Existing processes keep their injected environment until recreated. Restarting the rollout makes new containers read version-two. The bad readiness port prevents the surge Pod from becoming available; the two old replicas remain, and the Deployment eventually reports a stalled rollout. Undo repairs the template but leaves the ConfigMap at version-two. Explicit configuration restoration plus a consumer rollout returns version-one.

Step 2: The object changes to version-two while the existing UIDs still return version-one.

Step 3: New Pods consume version-two. Wait for old terminating Pods to disappear before treating the response set as the final two consumers.

Step 4: The new Pod stays unready at port 9999, while old available Pods remain. Even the misprobed process can answer direct HTTP on 8080. The rollout reports failure rather than automatically rolling back.

Step 5: The good readiness port is restored, but the separate ConfigMap still says version-two.

Step 6: Two Ready consumers return version-one and the new rollout completes.

Understanding check: No. That revision still refers to the same ConfigMap key. Newly created containers read its current value; the ConfigMap must be restored separately or configurations versioned under distinct names.